

Foldseek: The Missing Manual

October 13, 2025

Contents

1	Introduction to Foldseek Core Concepts	12
1.1	Why is Foldseek so fast? The 3Di Revolution	12
1.1.1	The 3Di Alphabet: Two-Tier System	12
1.1.1.1	Internal Classification (20 States)	12
1.1.1.2	External Representation (3-Letter Alphabet)	12
1.1.1.3	Internal vs. Output Representation	12
1.1.1.4	AA and 3Di Integration	12
1.1.2	Multi-Stage Structural Search Strategy	13
1.2	The Foldseek Database	13
1.3	Structural Similarity Measures	13
1.3.1	3Di Score	13
1.3.2	TM-Score	13
1.3.3	LDDT Score	13
1.4	The Power of Modularity: Building Structural Workflows	13
1.4.1	Example: Custom Structure-Based Clustering	13
1.4.2	Integration with External Tools	14
1.5	Foldseek vs MMseqs2: Key Differences	14
1.5.1	Search Methodology Comparison	14
1.5.2	Alignment Algorithm Comparison	14
1.6	Applications of Foldseek	15
1.6.1	Large-Scale Structure Comparison	15
1.6.2	Structure-Based Function Prediction	15
1.6.3	Drug Discovery	15
2	Foldseek User Guide	15
2.1		15
2.2	Summary	15
2.3	System requirements	15
2.3.0.1	Check system requirements under Linux	16
2.3.0.2	Check system requirements under macOS	16
2.4	Memory requirements	16
2.5	Installation	16
2.5.0.1	Compile from source under Linux	17
2.5.0.2	Compile from source for Linux with GPU support	17
2.5.0.3	Compile from source under macOS	18
2.5.0.3.1	Compiling under Clang	18
2.5.0.3.2	Compiling using GCC	18
2.5.1	Customizing compilation through CMake	18
2.5.1.1	Enable Google Cloud Storage support for createdb	18
2.6	Frequently Asked Questions	18
2.6.0.1	What is the hit probability?	18
2.6.0.2	How to apply U and T to a PDB file	19

2.6.1	Efficient pairwise alignment of given PDB pairs	19
3	Foldseek User Manual	20
3.1	Common Command Line Arguments	20
3.2	Modules	21
3.2.1	Easy Workflows	21
3.2.2	Structure Search Modules	21
3.2.3	Structure Clustering Modules	21
3.2.4	Multimer Analysis Modules	21
3.2.5	Structure Manipulation Modules	21
3.2.6	Database Management Modules	21
3.3	Easy Workflows	21
3.4	Structure Search Modules	21
3.5	Structure Clustering Modules	21
3.6	Multimer Analysis Modules	22
3.7	Structure Manipulation Modules	22
3.8	Database Management Modules	22
3.9	Output Formats	22
3.9.1	Tab-Separated Format (Default)	22
3.9.2	Custom Output Formats	22
3.9.3	HTML Output	22
3.9.4	PDB Superposition	22
3.10	Performance Optimization	22
3.10.1	GPU Acceleration	22
3.10.2	Memory Optimization	23
3.10.3	Sensitivity vs Speed Trade-offs	23
3.11	Integration with External Tools	23
3.11.1	Converting Results to Other Formats	23
3.11.2	Using with Structure Prediction	23
4	Easy Workflows	23
4.1	Easy Search Workflows	23
4.1.1	easy-search	23
4.1.2	easy-multimersearch	24
4.2	Easy Clustering Workflows	25
4.2.1	easy-cluster	25
4.2.2	easy-multimercluster	26
4.3	Easy RBH Workflow	27
4.3.1	easy-rbh	27
4.4	Output Files	28
4.4.1	Search Results	28
4.4.2	Clustering Results	28
4.4.3	Multimer Results	28
4.5	Performance Tips	28
4.5.1	Speed Optimization	28

4.5.2	Sensitivity Optimization	28
4.5.3	Memory Optimization	28
4.6	Integration Examples	28
4.6.1	With Structure Prediction	28
4.6.2	With Experimental Validation	29
4.6.3	Large-Scale Analysis	29
5	Structure Search Modules	29
5.1	Core Search Modules	29
5.1.1	search	29
5.1.2	structurealign	30
5.1.3	tmalign	31
5.1.4	structurerescorediagonal	32
5.2	Advanced Search Features	32
5.2.1	Iterative Search	32
5.2.2	Profile-Based Search	32
5.2.3	GPU-Accelerated Search	33
5.3	Search Output Formats	33
5.3.1	Tab-Separated Format (Default)	33
5.3.2	Custom Output with Structural Information	33
5.3.3	SAM Format	33
5.3.4	BLAST Format	33
5.4	Performance Optimization	33
5.4.1	Search Speed Optimization	33
5.4.2	Search Sensitivity Optimization	33
5.4.3	Memory Optimization	34
6	Structure Clustering Modules	34
6.0.1	cluster	34
6.0.2	clust	34
6.1	Clustering Algorithms	35
6.1.1	Set Cover Clustering (Mode 0)	35
6.1.2	Connected Component Clustering (Mode 1)	35
6.1.3	Greedy Incremental Clustering (Mode 2)	36
6.2	Clustering Output	36
6.2.1	Cluster Database Format	36
6.2.2	Tab-Separated Format	36
6.2.3	Representative Sequences	36
6.2.4	All Member Sequences	36
6.3	Advanced Clustering Features	37
6.3.1	Cascaded Clustering	37
6.3.2	Cluster Reassignment	37
6.3.3	Custom Clustering Criteria	37
6.4	Multimer Clustering	37
6.4.1	multimercluster	37

6.5	Clustering Evaluation	38
6.5.1	Cluster Quality Assessment	38
6.5.2	Cluster Size Distribution	38
6.6	Performance Optimization	38
6.6.1	Speed Optimization	38
6.6.2	Memory Optimization	39
6.6.3	Sensitivity Optimization	39
6.7	Integration Examples	39
6.7.1	With Structure Prediction	39
6.7.2	With Experimental Validation	39
6.7.3	Large-Scale Clustering	39
7	Multimer Analysis Modules	40
7.1	Core Multimer Modules	40
7.1.1	multimersearch	40
7.1.2	multimercluster	40
7.2	Multimer Scoring and Filtering	41
7.2.1	scoremultimer	41
7.2.2	filtermultimer	42
7.3	Multimer Result Processing	43
7.3.1	createmultimerreport	43
7.3.2	expandmultimer	43
7.4	Easy Multimer Workflows	43
7.4.1	easy-multimersearch	43
7.4.2	easy-multimercluster	44
7.5	Multimer Output Formats	45
7.5.1	Tab-Separated Format (Default)	45
7.5.2	Custom Complex Output	45
7.5.3	Complex Report Format	45
7.5.4	Multimer Cluster Format	45
7.6	Multimer Analysis Concepts	45
7.6.1	Complex TM-Score	45
7.6.2	Chain Assignment	46
7.6.3	Interface LDDT	46
7.7	Advanced Multimer Features	46
7.7.1	Iterative Multimer Search	46
7.7.2	Multimer Quality Assessment	46
7.7.3	Multimer Visualization	46
7.8	Performance Optimization	47
7.8.1	Speed Optimization	47
7.8.2	Memory Optimization	47
7.8.3	Sensitivity Optimization	47
7.9	Integration Examples	47
7.9.1	With Structure Prediction	47

	7.9.2	With Experimental Structures	47
	7.9.3	Large-Scale Analysis	47
8		Structure Manipulation Modules	48
	8.1	Database Creation and Management	48
		8.1.1 createdb	48
		8.1.2 compressca	49
	8.2	Format Conversion Modules	50
		8.2.1 convert2pdb	50
		8.2.2 convertalis	50
	8.3	Structure Analysis Modules	51
		8.3.1 aln2tmscore	51
		8.3.2 result2profile	52
	8.4	Structure Processing Workflows	53
		8.4.1 Structure Database Creation Pipeline	53
		8.4.2 Structure Conversion Pipeline	54
		8.4.3 Quality Assessment Pipeline	54
	8.5	Advanced Structure Manipulation	54
		8.5.1 Custom Structure Filtering	54
		8.5.2 Structure Profile Creation	54
		8.5.3 Batch Structure Processing	55
	8.6	Performance Optimization	55
		8.6.1 Database Creation Optimization	55
		8.6.2 Memory Optimization	55
		8.6.3 Speed Optimization	55
	8.7	Integration Examples	55
		8.7.1 With Structure Prediction	55
		8.7.2 With Experimental Validation	56
		8.7.3 Large-Scale Processing	56
9		Database Management Modules	56
	9.1	Database Download and Setup	56
		9.1.1 databases	56
	9.2	Index Creation and Management	58
		9.2.1 createindex	58
		9.2.2 createclusearchdb	59
	9.3	Database Operations	60
		9.3.1 Database Information and Utilities	60
	9.4	Custom Database Creation	60
		9.4.1 From Structure Files	60
		9.4.2 From Sequence Files with ProstT5	60
		9.4.3 From AlphaFold Results	61
	9.5	Database Maintenance	61
		9.5.1 Database Validation	61
		9.5.2 Database Cleanup	61

9.5.3	Database Backup	61
9.6	Performance Optimization	62
9.6.1	Index Optimization	62
9.6.2	Memory Optimization	62
9.6.3	GPU Optimization	62
9.7	Large-Scale Database Management	62
9.7.1	Batch Processing	62
9.7.2	Distributed Processing	62
9.7.3	Creating Searchable Databases from New Structures	63
9.8	Integration Examples	63
9.8.1	With High-Performance Computing	63
9.8.2	With Cloud Storage	63
9.8.3	With Version Control	63
9.9	Database Format Comparison with MMseqs2	64
9.9.1	Structural vs Sequence Databases	64
9.9.2	Database Creation Process	64
9.9.2.1	MMseqs2 Database Creation	64
9.9.2.2	Foldseek Database Creation	64
9.9.3	Index Structure Differences	65
9.9.3.1	MMseqs2 Index	65
9.9.3.2	Foldseek Index	65
9.9.4	Database Type Specifications	65
9.9.5	Performance Implications	66
9.9.5.1	Memory Efficiency	66
9.9.5.2	I/O Efficiency	66
9.9.5.3	Search Speed	66
9.9.6	Compatibility and Integration	66
9.9.6.1	Shared Database Operations	66
9.9.6.2	Unique Foldseek Operations	66
9.9.6.3	Cross-Compatibility	66
10	Foldseek Expert Manual: Advanced Database Interactions and Structural Alignment	66
10.1	Core Concepts	66
10.1.1	Header Database Handling Patterns	67
10.1.2	Performance Optimization Patterns	67
10.2	Module Documentation	68
10.2.1	createdb	68
10.2.2	structuresearch	68
10.2.3	structurealign	69
10.2.4	multimersearch	69
10.2.5	talign	69
10.2.6	convert2pdb	70
10.3	Advanced Database Operations	70
10.3.1	Index Creation and Management	70

10.3.1.1	Structural Index Architecture	70
10.3.1.2	Index File Structure	71
10.3.1.3	GPU-Optimized Databases	71
10.3.1.4	Database Splitting for Large-Scale Operations	71
10.3.1.5	Custom Database Creation	71
10.3.1.6	Database Concatenation	72
10.3.2	Database Performance Optimization	72
10.3.2.1	Memory Usage Optimization	72
10.3.2.2	Index Optimization Strategies	72
10.3.2.3	GPU Database Optimization	73
10.3.3	Database Quality Assessment	73
10.3.3.1	Structural Data Validation	73
10.3.3.2	Performance Benchmarking	74
10.3.4	Database Troubleshooting and Maintenance	74
10.3.4.1	Common Database Issues	74
10.3.4.2	Database Repair and Recovery	74
10.3.4.3	Performance Monitoring	75
10.3.4.4	Database Migration and Conversion	75
10.3.4.5	Database Maintenance Scripts	75
10.3.4.6	Database Backup and Recovery	76
10.3.4.7	Database Security	77
10.4	Structural Alignment Details	77
10.4.1	3Di Substitution Matrix	77
10.4.2	TM-score Calculation	77
10.4.3	LDDT Score Implementation	78
10.5	Performance Optimization	78
10.5.1	GPU Acceleration Details	78
10.5.1.1	CUDA Implementation	78
10.5.1.2	Memory Management	78
10.5.2	Parallel Processing	78
10.5.2.1	Multi-threading	78
10.5.2.2	Database Sharding	78
10.6	Advanced Workflows	78
10.6.1	Iterative Structural Refinement	78
10.6.2	Structure-Based Function Annotation	79
10.6.3	Quality Assessment Integration	79
10.7	Technical Comparison with MMseqs2	79
10.7.1	Database Architecture Differences	79
10.7.1.1	MMseqs2 Database Structure	79
10.7.1.2	Foldseek Database Structure	79
10.7.2	Algorithmic Differences	80
10.7.2.1	Prefiltering Stage	80
10.7.2.2	Alignment Stage	80

10.7.3	Performance Characteristics	80
10.7.3.1	Memory Efficiency	80
10.7.3.2	Speed Comparison	80
10.7.4	Data Processing Pipeline Differences	81
10.7.4.1	MMseqs2 Pipeline	81
10.7.4.2	Foldseek Pipeline	81
10.7.5	Advanced Features Comparison	82
10.7.5.1	MMseqs2 Advanced Features	82
10.7.5.2	Foldseek Advanced Features	82
10.7.6	Integration and Compatibility	82
10.7.6.1	Shared Infrastructure	82
10.7.6.2	Unique Integration Points	82
11	Foldseek Developer Manual: Architecture and Module Classification	82
11.1	Overview	82
11.2	Architecture Overview	83
11.2.1	Core Components	83
11.2.2	Data Flow Architecture	83
11.3	Module Classification	83
11.3.1	Easy Workflows	83
11.3.2	Structure Search Modules	83
11.3.3	Structure Clustering Modules	83
11.3.4	Multimer Analysis Modules	84
11.3.5	Structure Manipulation Modules	84
11.3.6	Database Management Modules	84
11.4	Hierarchical Dependencies	84
11.4.1	Easy Workflows → Core Modules	84
11.4.2	Core Modules → Structural Engines	84
11.4.3	Structural Engines → Data Access Layer	85
11.5	3Di Engine Architecture: Two-Tier System	85
11.5.1	3Di Conversion Pipeline	85
11.5.2	Secondary Structure Assignment	85
11.5.3	Two-Tier 3Di System	85
11.5.3.1	Tier 1: Internal Classification (20 States)	85
11.5.3.2	Tier 2: External Mapping (3-Letter Alphabet)	86
11.5.3.3	Internal vs. External Alphabet	86
11.5.4	AA and 3Di Integration: Combined Storage and Usage	87
11.5.4.1	Storage Architecture	87
11.5.4.2	Database File Structure	87
11.5.4.3	Combined Alignment Process	87
11.5.4.4	Key Implementation Details	88
11.5.5	SIMD Optimization	88
11.6	Structural Database Layer	88
11.6.1	Database File Architecture	88

11.6.1.1	Database File Components	89
11.6.1.2	Index Structure and Memory Mapping	89
11.6.1.3	Index File Format	90
11.6.1.4	Structural Index Architecture	90
11.6.1.5	Database Type Specifications	90
11.6.1.6	Memory Mapping Strategy	91
11.7	Alignment Engine Architecture	91
11.7.1	Multiple Alignment Algorithms	91
11.7.2	TM-score Implementation	91
11.7.3	SIMD Alignment Acceleration	91
11.8	Multimer Engine Architecture	92
11.8.1	Complex Representation	92
11.8.2	Chain Matching Algorithm	92
11.8.3	Interface Analysis	92
11.9	GPU Acceleration Architecture	93
11.9.1	CUDA Integration	93
11.9.2	Memory Management	93
11.10	Performance Optimization	93
11.10.1	Parallel Processing	93
11.10.2	Memory Optimization	93
11.10.3	Algorithmic Optimization	93
11.11	Development Guidelines	94
11.11.1	Code Organization	94
11.11.2	Testing Strategy	94
11.11.3	Performance Profiling	94

1 Introduction to Foldseek Core Concepts

Foldseek is a revolutionary software suite for fast and sensitive protein structure searching and clustering. It achieves unprecedented speed by representing protein structures using a compact 3D-interaction (3Di) alphabet, enabling structural comparisons at sequence-like speeds while maintaining high sensitivity.

1.1 Why is Foldseek so fast? The 3Di Revolution

The key innovation behind Foldseek's speed is the **3Di alphabet** - a compact representation that captures essential structural information while dramatically reducing the complexity of structural comparisons.

1.1.1 The 3Di Alphabet: Two-Tier System

Traditional structural alignment methods compare full 3D coordinates, which is computationally expensive. Foldseek uses a sophisticated **two-tier system** for structural representation:

1.1.1.1 Internal Classification (20 States)

- **20 structural centroids** (states 0-19) trained via neural network embedding
- Each residue is assigned to the closest centroid using Euclidean distance
- Provides maximum structural discrimination for accurate similarity detection
- **Used in:** Structure-to-structure alignment and initial classification

1.1.1.2 External Representation (3-Letter Alphabet)

- **H** (Helix): α -helical conformations
- **E** (Strand): β -strand conformations
- **C** (Coil): Loop and irregular conformations
- **Used in:** Database storage and all search operations

1.1.1.3 Internal vs. Output Representation

The 3Di system uses a sophisticated two-tier approach for structural classification:

1. **Internal Classification:** Uses **20 centroids** (states 0-19) trained on structural features to provide detailed structural discrimination
2. **Final Output:** Maps these 20 internal states to the **3-letter alphabet** (H/E/C) through a substitution matrix for efficient k-mer matching and alignment

This design enables: - **Rich structural classification** with 20 internal states for high sensitivity - **Compact representation** with 3-letter output for fast searching - **Memory efficiency** with ~1 byte per residue instead of ~8 bytes for coordinates - **10,000x faster** prefiltering compared to coordinate-based methods

1.1.1.4 AA and 3Di Integration

Foldseek stores and uses both amino acid (AA) sequences and 3Di sequences together:

- **AA sequences** (20-letter alphabet): Stored in database files for sequence-based operations
- **3Di sequences** (3-letter alphabet): Stored in database_ss files for structural operations
- **Combined alignment:** Uses both AA and 3Di scoring matrices for optimal sensitivity

- **Synchronized storage:** Both sequence types have identical structure and indexing

1.1.2 Multi-Stage Structural Search Strategy

Foldseek employs a hierarchical search strategy similar to sequence-based tools but optimized for structural data:

1. **3Di K-mer Matching:** Fast identification of structurally similar regions using 3Di k-mers
2. **Structural Prefiltering:** Double consecutive k-mer matches on the same diagonal identify promising candidates
3. **Ungapped Structural Alignment:** Fast verification of structural similarity
4. **Full Structural Alignment:** TM-align or 3Di+AA Smith-Waterman alignment for final scoring

1.2 The Foldseek Database

High-level overview: Foldseek uses compact 3Di sequences plus optional coordinates to enable fast structural search. For file layout and operational details, see: - Databases (foldseek_docs/submodules/databases.md) - Developer Manual (foldseek_docs/developer_manual.md)

1.3 Structural Similarity Measures

1.3.1 3Di Score

The primary scoring mechanism uses 3Di sequences with position-specific scoring matrices optimized for structural similarity detection.

1.3.2 TM-Score

For high-precision alignment, Foldseek can use TM-score, which provides: - **Global alignment** quality assessment - **Statistical significance** estimates - **Superposition** capabilities for visualization

1.3.3 LDDT Score

Local Distance Difference Test (LDDT) provides residue-level quality assessment, particularly useful for: - **Local structural conservation** analysis - **Domain identification** - **Quality assessment** of predicted structures

1.4 The Power of Modularity: Building Structural Workflows

Foldseek inherits MMseqs2's modular architecture, allowing users to build custom structural analysis pipelines:

1.4.1 Example: Custom Structure-Based Clustering

```
# Create structural database
foldseek createdb structures/ structDB

# Perform structural search
foldseek search structDB structDB aln tmp

# Convert to structural clustering
foldseek cluster structDB structDB aln structDB_clu tmp
```

```
# Extract representatives
foldseek createsubdb structDB_clu structDB structDB_reps
foldseek convert2fasta structDB_reps structDB_reps.fasta
```

1.4.2 Integration with External Tools

The modular design enables seamless integration with other structural biology tools:

```
# Extract cluster members
foldseek createseqfiledb structDB structDB_clu structDB_members

# Run external analysis on each cluster
foldseek apply structDB_members structDB_analysis -- your_analysis_tool -i -
```

1.5 Foldseek vs MMseqs2: Key Differences

While Foldseek builds upon the MMseqs2 framework, it introduces fundamental differences in search methodology, database structure, and alignment algorithms to enable efficient structural comparisons.

1.5.1 Search Methodology Comparison

Aspect	MMseqs2	Foldseek
Core Algorithm	K-mer based sequence similarity search	3Di-based structural similarity search
Alphabet	20 amino acids (reduced alphabet for sensitivity)	3-letter 3Di alphabet (H/E/C) with 20 internal centroids
Search Strategy	1. K-mer matching 2. Double consecutive k-mer filtering 3. Ungapped alignment 4. SIMD-accelerated Smith-Waterman	1. 3Di k-mer matching 2. Structural prefiltering 3. Ungapped structural alignment 4. 3Di+AA or TM-align
Speed Advantage	10,000x faster than BLAST	10,000x faster than coordinate-based methods
Memory Usage	~7 bytes per residue for index	~1 byte per residue for 3Di sequences

1.5.2 Alignment Algorithm Comparison

Feature	MMseqs2	Foldseek
Primary Algorithm	Smith-Waterman sequence alignment	3Di+AA combined alignment
Scoring	Sequence identity, E-values	TM-score, LDDT, 3Di scores

Feature	MMseqs2	Foldseek
Structural Alignment	Not available	TM-align integration
GPU Support	Yes (sequence search)	Yes (structural search)
Substitution Matrix	BLOSUM62, VTML matrices	3Di substitution matrix

1.6 Applications of Foldseek

1.6.1 Large-Scale Structure Comparison

- **AlphaFold Database** screening
- **PDB-wide** structural similarity searches
- **Metagenomic** structure annotation

1.6.2 Structure-Based Function Prediction

- **SCOPe/CATH** domain classification
- **EC number** assignment via structural similarity
- **GO term** prediction from structural neighbors

1.6.3 Drug Discovery

- **Binding site** identification
- **Off-target** effect prediction
- **Lead optimization** through structural similarity

This modular, efficient approach makes Foldseek not just a tool, but a comprehensive platform for structural bioinformatics research.

2 Foldseek User Guide

2.1

- Summary
- System requirements
- Memory requirements
- Installation
 - Customizing compilation through CMake
- Frequently Asked Questions
 - What is the hit probability?
 - How to apply U and T to a PDB file
 - Efficient pairwise alignment of given PDB pairs

2.2 Summary

2.3 System requirements

Foldseek runs on modern UNIX operating systems and is tested on Linux and macOS. Additionally, we are providing a preview version for Windows.

Foldseek takes advantage of multi-core systems through OpenMP and uses the SIMD capabilities of the system. Optimal performance requires a system supporting the AVX2 instruction set, however SSE4.1 and very old systems with SSE2 are also supported. It also supports the PPC64LE and ARM64 processor architectures, these require support for the AltiVec or NEON SIMD instruction sets, respectively.

To check if Foldseek supports your system execute the following commands, depending on your operating system:

2.3.0.1 Check system requirements under Linux

```
[ $(uname -m) = "x86_64" ] && echo "64bit: Yes" || echo "64bit: No"
grep -q avx2 /proc/cpuinfo && echo "AVX2: Yes" || echo "AVX2: No"
grep -q sse4_1 /proc/cpuinfo && echo "SSE4.1: Yes" || echo "SSE4.1: No"
# for very old systems which support neither SSE4.1 or AVX2
grep -q sse2 /proc/cpuinfo && echo "SSE2: Yes" || echo "SSE2: No"
```

2.3.0.2 Check system requirements under macOS

```
[ $(uname -m) = "x86_64" ] && echo "64bit: Yes" || echo "64bit: No"
sysctl machdep.cpu.leaf7_features | grep -q AVX2 && echo "AVX2: Yes" || echo
"AVX2: No"
sysctl machdep.cpu.features | grep -q SSE4.1 && echo "SSE4.1: Yes" || echo
"SSE4.1: No"
```

2.4 Memory requirements

To ensure optimal performance of the software, it is important to have a machine with adequate memory (RAM) capacity. The required memory can be calculated using the following formula:

RAM Needed = (6 bytes C α + 1 3Di byte + 1 AA byte) * (residues in the database).

For example, for the AFDB50 dataset, the memory requirement can be calculated as:

8 byte * 54*10⁶ (Seqs) * 350 (avg. protein length) = 151G

If searching with 3Di/AA without using the `--sort-by-structure-bits 0` option, the C α information can be disregarded. This would reduce the memory requirement for the AFDB50 dataset to:

2 bytes * 54 x 10⁶ (sequences) * 350 (average protein length) = 35 GB

Please note that disabling the `--sort-by-structure-bits 0` option affects the final score and ranking of hits, but not the E-values themselves. Ranking alterations primarily occur for E-values less than 10⁻¹.

2.5 Installation

Foldseek can be installed for Linux or macOS

(1) downloading a statically compiled version For Linux computer with supports AVX2 use:

```
wget https://mmseqs.com/foldseek/foldseek-linux-avx2.tar.gz
tar xvfz foldseek-linux-avx2.tar.gz
export PATH=$(pwd)/foldseek/bin/:$PATH
```


Linux with SSE4.1

```
wget https://mmseqs.com/foldseek/foldseek-linux-sse41.tar.gz
tar xvzf foldseek-linux-sse41.tar.gz
export PATH=$(pwd)/foldseek/bin/:$PATH
```

macOS build (universal binary with SSE4.1/AVX2/M1 NEON)

```
wget https://mmseqs.com/foldseek/foldseek-osx-universal.tar.gz
tar xvzf foldseek-osx-universal.tar.gz
export PATH=$(pwd)/foldseek/bin/:$PATH
```

(2) using bioconda

```
conda install -c conda-forge -c bioconda foldseek
```

(3) compiling the from source (see below),

2.5.0.1 Compile from source under Linux

Compiling Foldseek from source has the advantage that it will be optimized to the specific system, which should improve its performance. To compile Foldseek git, g++ (4.9 or higher) and cmake (2.8.12 or higher) are needed. Afterwards, the foldseek binary will be located in build/bin/.

```
git clone https://github.com/steineggerlab/foldseek.git
cd foldseek
mkdir build
cd build
cmake -DCMAKE_BUILD_TYPE=RELEASE -DCMAKE_INSTALL_PREFIX=. . .
make
make install
export PATH=$(pwd)/bin/:$PATH
```

See the Customizing compilation through CMake section if you compile Foldseek on a different system than the one where it will eventually run.

2.5.0.2 Compile from source for Linux with GPU support

Foldseek supports GPU-accelerated protein structure search and ProstT5-based 3Di prediction. This requires an NVIDIA GPU of the Ampere generation or newer for full speed, however, also works at reduced speed for Tesla-generation GPUs. We recommend to install CUDA (>= 12.4) dependencies through Conda. The command below compiles code for multiple GPU generations. To speed up compilation, you can use `-DCMAKE_CUDA_ARCHITECTURES="native"`, to compile the code for the installed GPUs only.

```
conda create -n nvcc -c conda-forge cuda-nvcc cuda-cudart-dev libcublas-dev
libcublas-static cuda-version=12.6 cmake
conda activate nvcc
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=RELEASE -DCMAKE_INSTALL_PREFIX=. -DENABLE_CUDA=1 -
DCMAKE_CUDA_ARCHITECTURES="75;80;86;89;90" . .
make -j8
```

```
make install
export PATH=$(pwd)/bin/:$PATH
```

2.5.0.3 Compile from source under macOS

2.5.0.3.1 Compiling under Clang

To compile Foldseek with (Apple-)Clang you need to install either XCode or the Command Line Tools. You also need libomp. We recommend installing it using Homebrew:

```
brew install cmake libomp zlib bzip2
```

CMake currently does not correctly identify paths to libomp. Use the script in util/build_osx.sh to compile Foldseek. The resulting binary will be placed in OUTPUT_DIR/mmseqs.

```
./util/build_osx.sh PATH_TO_FOLDSEEK_REPO OUTPUT_DIR
```

2.5.0.3.2 Compiling using GCC

Please install the following packages with Homebrew:

```
brew install cmake gcc@11 zlib bzip2
```

Use the following cmake call:

```
CC="gcc-14" CXX="g++-14" cmake -DCMAKE_BUILD_TYPE=RELEASE -
DCMAKE_INSTALL_PREFIX=. ..
```

2.5.1 Customizing compilation through CMake

Most of the MMseqs2 CMake options also apply to Foldseek, refer to MMseqs2's user guide for details.

2.5.1.1 Enable Google Cloud Storage support for createdb

Install the google-cloud-cpp package from vcpkg:

```
git clone https://github.com/microsoft/vcpkg.git
./vcpkg/bootstrap-vcpkg.sh
./vcpkg/vcpkg install google-cloud-cpp
```

Foldseek can now be compiled with GCS support:

```
cd path-to-foldseek
mkdir build && cd build
cmake -DHAVE_GCS=1 -DCMAKE_TOOLCHAIN_FILE=[path to vcpkg]/scripts/buildsystems/
vcpkg.cmake ..
make -j $(nproc --all)
```

2.6 Frequently Asked Questions

2.6.0.1 What is the hit probability?

Foldseek computes for each match a simple estimate for the probability that the match is a true positive match given its structural bit score. Here, hits within the same superfamily are TP, hits to another fold are FP, and hits to the same family or to another superfamily are ignored. The

probability is the fraction of TP hits from TP and FP hits found at the score on average. For this, we estimate the bit score distributions of TP and FP hits. Both score distributions were fitted on SCOPe40. For example, Foldseek finds around the same number of FP and TP with a score of 51 in SCOPe40. The probability for a hit with score 51 is therefore 50%.

2.6.0.2 How to apply U and T to a PDB file

In Foldseek we apply U and T to the target to superposition it onto the query structure. Following is some awk one-liner snippet that applies the rotations to an input PDB file. You have to provide UT as a vector of 12 values, 9 being the U matrix and 3 the T vector.

```
awk -v
UT="-0.672446,0.740134,-0.004138,-0.740140,-0.672409,0.007633,0.002867,0.008196,0.999962,0.099348,-
'BEGIN {split(UT, arr, ",")} {
    if ($0 ~ /^ATOM|^HETATM/) {
        x = $7
        y = $8
        z = $9
        x_new = (x * arr[1] + y * arr[2] + z * arr[3]) + arr[10]
        y_new = (x * arr[4] + y * arr[5] + z * arr[6]) + arr[11]
        z_new = (x * arr[7] + y * arr[8] + z * arr[9]) + arr[12]
        printf "%-6s%d %-4s%3s %s%4d %8.3f%8.3f%8.3f%6.2f%6.2f
%2s\n", $1, $2, $3, $4, $5, $6, x_new, y_new, z_new, $11, $12, $13
    }
    else {
        print $0
    }
}' input.pdb > output.pdb
```

2.6.1 Efficient pairwise alignment of given PDB pairs

You can make a your own prefiltering database to tell the structurealign module what pairs to align.

```
# assuming you have a query and target database
foldseek createdb inputs1/ db1
foldseek createdb inputs2/ db2

# make a mapping of the accession that you want to align (check 2nd column in the
dbN.lookup file)
echo -e "d1asha_\td1b0ba_\nd1asha_\td1cg5a_\n" > to_align.tsv

# convert this into the internal numeric database keys
awk 'FNR == 1 { findex++; } \
    findex == 1 { f1[$2] = $1; next; } \
    findex == 2 { f2[$2] = $1; next; } \
    $1 in f1 && $2 in f2 { print f1[$1]"\t"f2[$2]; }' \
    db1.lookup db2.lookup <(sort -s -k1,1n to_align.tsv ) > keys.tsv

# make a fake prefiltering database
foldseek tsv2db keys.tsv pref --output-dbtype 7
```

```
# foldseek alignment
foldseek structurealign db1 db2 pref aln
```

```
# m8 human readable output
foldseek convertalis db1 db2 aln aln.m8
```

3 Foldseek User Manual

This document provides a detailed manual for each module in Foldseek, the fast and sensitive protein structure search and clustering tool.

3.1 Common Command Line Arguments

The following are some of the most common command line arguments used across various Foldseek modules.

Flag	Description	Default
-s <float>	Sensitivity parameter for structural search (higher is more sensitive).	9.5
-c <float>	Coverage threshold for clustering and alignment.	0.0
--cov-mode <int>	Coverage mode (0: coverage of query and target, 1: coverage of target, 2: coverage of query).	0
--min-seq-id <float>	Minimum sequence identity for clustering and alignment.	0.0
--alignment-type <int>	Alignment type (0: 3Di only, 1: TAlign, 2: 3Di+AA).	2
--tmscore-threshold <float>	TM-score threshold for accepting alignments.	0.0
--lddt-threshold <float>	LDDT threshold for accepting alignments.	0.0
--threads <int>	Number of threads to use.	1
-v <int>	Verbosity level (0: quiet, 1: default, 2: verbose, 3: debug).	1
<tmpDir>	A temporary directory for intermediate files.	

For more specific parameters, please refer to the documentation for each module.

3.2 Modules

Foldseek is composed of many different modules that can be combined to create powerful structural analysis workflows. The modules are grouped by their functionality below.

- Easy Workflows
- Structure Search
- Structure Clustering
- Multimer Analysis
- Structure Manipulation
- Database Management

3.2.1 Easy Workflows

High-level workflows for common tasks. See `foldseek_docs/submodules/easy_workflows.md` for full usage, parameters, and examples.

3.2.2 Structure Search Modules

Core search and alignment modules. See `foldseek_docs/submodules/structure_search.md` for details. (Note: `aln2tmscore` and `convert2pdb` are documented under structure manipulation.)

3.2.3 Structure Clustering Modules

Clustering algorithms and workflows. See `foldseek_docs/submodules/structure_clustering.md`.

3.2.4 Multimer Analysis Modules

Complex (multimer) search, clustering, and reporting. See `foldseek_docs/submodules/multimer.md`.

3.2.5 Structure Manipulation Modules

Database creation, conversion, and post-processing. See `foldseek_docs/submodules/structure_manipulation.md`.

3.2.6 Database Management Modules

Database downloads, indexing, and cluster search DBs. See `foldseek_docs/submodules/databases.md`.

3.3 Easy Workflows

See `foldseek_docs/submodules/easy_workflows.md` for full usage, parameters, and examples for `easy-search`, `easy-cluster`, `easy-rbh`, `easy-multimersearch`, and `easy-multimercluster`.

3.4 Structure Search Modules

See `foldseek_docs/submodules/structure_search.md` for search and alignment modules: `search`, `structurealign`, `talign`, and `structurerescorediagonal`.

3.5 Structure Clustering Modules

See `foldseek_docs/submodules/structure_clustering.md` for cluster and `clust` usage and algorithms.

3.6 Multimer Analysis Modules

See `foldseek_docs/submodules/multimer.md` for multimer search, clustering, scoring, filtering, and reports.

3.7 Structure Manipulation Modules

See `foldseek_docs/submodules/structure_manipulation.md` for `createdb`, `compressca`, `convertalis`, `aln2tmscore`, `result2profile`, and `convert2pdb`.

3.8 Database Management Modules

See `foldseek_docs/submodules/databases.md` for `databases`, `createindex`, and `createclusearchdb`.

3.9 Output Formats

3.9.1 Tab-Separated Format (Default)

The default output format includes columns for query, target, sequence identity, alignment length, mismatches, gap openings, start/end positions, E-value, and bit score.

3.9.2 Custom Output Formats

Use `--format-output` to specify custom columns:

```
foldseek easy-search query target result tmp --format-output  
"query,target,fident,alntmscore,u,t"
```

Available Fields: - query: Query identifier - target: Target identifier - fident: Sequence identity - alnlen: Alignment length - qstart, qend: Query start/end positions - tstart, tend: Target start/end positions - eval: E-value - bits: Bit score - alntmscore: TM-score of alignment - qtmscore: TM-score normalized by query - ttmscore: TM-score normalized by target - lddt: Average LDDT - prob: Hit probability - u: Rotation matrix - t: Translation vector

3.9.3 HTML Output

Interactive HTML visualization:

```
foldseek easy-search query target result.html tmp --format-mode 3
```

3.9.4 PDB Superposition

Superimposed PDB files:

```
foldseek easy-search query target result tmp --format-mode 5
```

3.10 Performance Optimization

3.10.1 GPU Acceleration

Enable GPU acceleration for faster searches:

```
foldseek easy-search query target result tmp --gpu 1 --prefilter-mode 1
```

3.10.2 Memory Optimization

For large databases, consider: - Using `--sort-by-structure-bits 0` to reduce memory usage - Splitting large searches into smaller chunks - Using precomputed indexes for repeated searches

3.10.3 Sensitivity vs Speed Trade-offs

- Lower `-s` values: Faster but less sensitive
- Higher `-s` values: Slower but more sensitive
- Use `--exhaustive-search` for maximum sensitivity (very slow)
- Use `--num-iterations` for iterative refinement

3.11 Integration with External Tools

3.11.1 Converting Results to Other Formats

Convert alignment results to various formats

```
foldseek convertalis queryDB targetDB alignmentDB results.tsv --format-output query,target,fident,alnlen,evalue,bits
```

Create profiles from search results

```
foldseek result2profile queryDB targetDB search_results profileDB
```

3.11.2 Using with Structure Prediction

Predict structures with ColabFold

```
colabfold_batch sequences.fasta structures/
```

Search predicted structures

```
foldseek easy-search structures/ targetDB results tmp
```

4 Easy Workflows

Foldseek provides several easy-to-use workflows that combine multiple modules for common structural analysis tasks. These workflows are designed to be user-friendly while providing access to advanced structural comparison capabilities.

4.1 Easy Search Workflows

4.1.1 easy-search

Description: Fast and sensitive protein structure search against a target database.

Usage:

```
foldseek easy-search queryStructures targetDB resultDB tmpDir [options]
```

Parameters:

Parameter	Description	Default
<code>-s <float></code>	Sensitivity (1.0-9.5, higher = more sensitive)	9.5

Parameter	Description	Default
--alignment-type <int>	0=3Di only, 1=TMalign, 2=3Di+AA	2
--format-mode <int>	Output format (0=tab, 3=HTML, 5=PDB)	0
--format-output <string>	Custom output columns	query,target,fi- dent,alnlen,mis- match,gapopen,qstart,qend,tstart,tend,evalue
-e <float>	E-value threshold	0.001
--max-seqs <int>	Maximum sequences per query	1000
--num-iterations <int>	Number of search iterations	0
--exhaustive-search	Skip prefilter (slower but more sensitive)	false
--cluster-search <int>	For clustered DBs: 0=representatives only, 1=all members	0
--gpu <int>	Enable GPU acceleration	0
--prefilter-mode <int>	Prefilter mode (1=GPU-optimized)	0

Examples:

```
# Basic structural search
foldseek easy-search query.pdb targetDB results tmp

# High sensitivity search with TM-align
foldseek easy-search query.pdb targetDB results tmp -s 9.5 --alignment-type 1

# GPU-accelerated search
foldseek easy-search query.pdb targetDB results tmp --gpu 1 --prefilter-mode 1

# Custom output format
foldseek easy-search query.pdb targetDB results tmp --format-output
"query,target,alntmscore,u,t"

# HTML visualization output
foldseek easy-search query.pdb targetDB results.html tmp --format-mode 3
```

4.1.2 easy-multimersearch

Description: Search protein complexes (multimers) against a target database.

Usage:

```
foldseek easy-multimersearch queryComplexes targetDB resultDB tmpDir [options]
```

Parameters:

Parameter	Description	Default
--multimer-tm-threshold <float>	Complex TM-score threshold	0.0
--chain-tm-threshold <float>	Individual chain TM-score threshold	0.0
--interface-lddt-threshold <float>	Interface LDDT threshold	0.0
--alignment-type <int>	Alignment algorithm	2
-e <float>	E-value threshold	0.001
--format-output <string>	Custom output columns	query,target,fi- dent,alnlen,mis- match,gapopen,qstart,qend,tstart,tend,evalue plexassignid

Examples:

```
# Basic multimer search
foldseek easy-multimersearch complex.pdb targetDB results tmp

# Strict complex matching
foldseek easy-multimersearch complex.pdb targetDB results tmp \
  --multimer-tm-threshold 0.8 --chain-tm-threshold 0.6 --interface-lddt-threshold 0.7

# Custom output with complex scores
foldseek easy-multimersearch complex.pdb targetDB results tmp \
  --format-output "query,target,complexqtm_score,complexttm_score,complexassignid"
```

4.2 Easy Clustering Workflows

4.2.1 easy-cluster

Description: Fast protein structure clustering using structural similarity.

Usage:

```
foldseek easy-cluster inputStructures resultDB tmpDir [options]
```

Parameters:

Parameter	Description	Default
-c <float>	Coverage threshold	0.0
--min-seq-id <float>	Minimum sequence identity	0.0
--alignment-type <int>	0=3Di only, 1=TMalign, 2=3Di+AA	2

Parameter	Description	Default
--tmscore-threshold <float>	TM-score threshold	0.0
--lddt-threshold <float>	LDDT threshold	0.0
--cov-mode <int>	0=bidirectional, 1=target, 2=query	0
--cluster-mode <int>	0=set cover, 1=connected component, 2=greedy	0
-e <float>	E-value threshold	0.001

Examples:

```
# Basic structural clustering
foldseek easy-cluster structures/ cluster_results tmp

# High-quality clustering with TM-align
foldseek easy-cluster structures/ cluster_results tmp --alignment-type 1 --
tmscore-threshold 0.5

# Clustering with specific coverage requirements
foldseek easy-cluster structures/ cluster_results tmp -c 0.8 --cov-mode 0
```

4.2.2 easy-multimercluster

Description: Cluster protein complexes based on structural similarity.

Usage:

```
foldseek easy-multimercluster inputComplexes resultDB tmpDir [options]
```

Parameters:

Parameter	Description	Default
--multimer-tm-threshold <float>	Complex TM-score threshold	0.65
--chain-tm-threshold <float>	Individual chain TM-score threshold	0.5
--interface-lddt-threshold <float>	Interface LDDT threshold	0.65
--cov-mode <int>	Coverage mode	0
--cluster-mode <int>	Clustering algorithm	0
-e <float>	E-value threshold	0.001

Examples:

```
# Basic multimer clustering
foldseek easy-multimercluster complexes/ multimer_clusters tmp

# Strict multimer clustering
foldseek easy-multimercluster complexes/ multimer_clusters tmp \
  --multimer-tm-threshold 0.8 --chain-tm-threshold 0.7 --interface-lddt-threshold
0.8
```

4.3 Easy RBH Workflow

4.3.1 easy-rbh

Description: Reciprocal best hit analysis for structural data.

Usage:

```
foldseek easy-rbh queryFastaFile1 targetFastaFile|targetDB alignmentFile tmpDir
[options]
```

Parameters:

Parameter	Description	Default
-s <float>	Sensitivity (1.0-9.5, higher = more sensitive)	4.0
--alignment-type <int>	0=3Di only, 1=TMalign, 2=3Di+AA	2
-e <float>	E-value threshold	10.0
--max-seqs <int>	Maximum sequences per query	300
--format-mode <int>	Output format (0=tab, 3=HTML, 5=PDB)	0
--format-output <string>	Custom output columns	query,target,fi- dent,alnlen,mis- match,gapopen,qstart,qend,tstart,tend,evaluate

Examples:

```
# Basic reciprocal best hit analysis
foldseek easy-rbh query.fasta target.fasta rbh_results tmp

# High sensitivity RBH analysis
foldseek easy-rbh query.fasta target.fasta rbh_results tmp -s 9.5 --alignment-
type 1

# Custom output format
foldseek easy-rbh query.fasta target.fasta rbh_results tmp --format-output
"query,target,alntmscore,u,t"

# HTML visualization output
foldseek easy-rbh query.fasta target.fasta rbh_results.html tmp --format-mode 3
```

4.4 Output Files

Easy workflows generate multiple output files:

4.4.1 Search Results

- resultDB: Main result database
- resultDB.index: Result index
- resultDB_report: Search report (if applicable)

4.4.2 Clustering Results

- resultDB_clu: Cluster assignments
- resultDB_clu.index: Cluster index
- resultDB_rep_seq.fasta: Representative sequences
- resultDB_all_seq.fasta: All sequences with cluster markers
- resultDB_cluster.tsv: Tab-separated cluster assignments

4.4.3 Multimer Results

- resultDB: Complex alignment results
- resultDB_report: Complex matching report
- resultDB_cluster.tsv: Multimer cluster assignments

4.5 Performance Tips

4.5.1 Speed Optimization

- Use lower -s values for faster searches (e.g., -s 7.5)
- Enable GPU acceleration with --gpu 1 --prefilter-mode 1
- Use --max-seqs to limit results per query
- Use --exhaustive-search only when maximum sensitivity is required

4.5.2 Sensitivity Optimization

- Use higher -s values for distant homology detection
- Enable --num-iterations for iterative refinement
- Use --alignment-type 1 for TM-align based scoring
- Adjust -e threshold based on your requirements

4.5.3 Memory Optimization

- Use --sort-by-structure-bits 0 to reduce memory usage
- Split large searches into smaller batches
- Use precomputed indexes for repeated searches
- Monitor memory usage with -v 2

4.6 Integration Examples

4.6.1 With Structure Prediction

```
# Predict structures with ColabFold
colabfold_batch sequences.fasta predicted_structures/
```

```
# Search predicted structures
foldseek easy-search predicted_structures/ reference_db results tmp
```

4.6.2 With Experimental Validation

```
# Search against experimental structures
foldseek easy-search predicted_model.pdb experimental_db validation_results tmp

# Filter high-confidence matches
foldseek filterdb validation_results high_confidence --filter-expression '$I1 < 1e-10'
```

4.6.3 Large-Scale Analysis

```
# Process large datasets in batches
for i in {1..100}; do
    foldseek easy-search batch_${i}.pdb large_db results_${i} tmp_${i} --max-seqs 100
done

# Merge results
foldseek mergedbs all_results results_* tmp
```

These easy workflows provide a user-friendly interface to Foldseek’s powerful structural analysis capabilities while maintaining access to advanced parameters for fine-tuning performance and sensitivity.

5 Structure Search Modules

Foldseek provides several modules for performing structural similarity searches, from high-level workflows to low-level alignment algorithms.

5.1 Core Search Modules

5.1.1 search

Description: Main structural search module combining prefiltering and alignment.

Usage:

```
foldseek search queryDB targetDB resultDB tmpDir [options]
```

Parameters:

Parameter	Description	Default
-s <float>	Sensitivity (1.0-9.5, higher = more sensitive)	9.5
--alignment-type <int>	0=3Di only, 1=TMalign, 2=3Di+AA	2
--max-seqs <int>	Maximum sequences per query	1000
-e <float>	E-value threshold	10.0

Parameter	Description	Default
--max-accept <int>	Maximum accepted alignments per query	2147483647
--max-rejected <int>	Maximum consecutive rejections before stopping	2147483647
--prefilter-mode <int>	0=3Di k-mer, 1=ungapped alignment	0
--diag-score <bool>	Enable diagonal score computation	true
--min-ungapped-score <int>	Minimum ungapped alignment score	30
--comp-bias-corr <int>	Compositional bias correction	1
--mask <int>	Low complexity masking	0
--mask-prob <float>	Masking probability threshold	1.0
--gpu <int>	Enable GPU acceleration	0

Examples:

```
# Basic structural search
foldseek search queryDB targetDB results tmp

# High sensitivity search
foldseek search queryDB targetDB results tmp -s 9.5 --alignment-type 1

# Fast search with lower sensitivity
foldseek search queryDB targetDB results tmp -s 7.5 --max-seqs 300

# GPU-accelerated search
foldseek search queryDB targetDB results tmp --gpu 1 --prefilter-mode 1
```

5.1.2 structurealign

Description: Direct structural alignment between two structure sets.

Usage:

```
foldseek structurealign queryDB targetDB resultDB [options]
```

Parameters:

Parameter	Description	Default
--alignment-type <int>	0=3Di only, 1=TMalign, 2=3Di+AA	2
--tmscore-threshold <float>	TM-score threshold	0.0
--lddt-threshold <float>	LDDT threshold	0.0

Parameter	Description	Default
<code>--max-seqs <int></code>	Maximum sequences per query	1000
<code>-e <float></code>	E-value threshold	0.001
<code>--comp-bias-corr <int></code>	Compositional bias correction	1
<code>--mask <int></code>	Low complexity masking	1

Examples:

```
# Direct structural alignment
foldseek structurealign queryDB targetDB alignments

# TM-align based alignment
foldseek structurealign queryDB targetDB alignments --alignment-type 1 --tmscore-
threshold 0.5

# High-quality alignments only
foldseek structurealign queryDB targetDB alignments --lddt-threshold 0.7
```

5.1.3 talign

Description: TM-align based structural alignment.

Usage:

```
foldseek talign queryDB targetDB resultDB [options]
```

Parameters:

Parameter	Description	Default
<code>--tmscore-threshold <float></code>	TM-score threshold	0.0
<code>--tmscore-threshold-mode <int></code>	0=alignment, 1=query, 2=target	0
<code>--max-seqs <int></code>	Maximum sequences per query	1000
<code>-e <float></code>	E-value threshold	0.001

Examples:

```
# TM-align with default settings
foldseek talign queryDB targetDB tm_results

# Strict TM-score threshold
foldseek talign queryDB targetDB tm_results --tmscore-threshold 0.6

# Query-normalized TM-scores
foldseek talign queryDB targetDB tm_results --tmscore-threshold-mode 1
```

5.1.4 structurerescorediagonal

Description: Structure-based rescoring of diagonals from prefiltering.

Usage:

```
foldseek structurerescorediagonal queryDB targetDB prefilterDB resultDB [options]
```

Parameters:

Parameter	Description	Default
--alignment-type <int>	0=3Di only, 1=TMalign, 2=3Di+AA	2
--tmscore-threshold <float>	TM-score threshold	0.0
--lddt-threshold <float>	LDDT threshold	0.0
-e <float>	E-value threshold	10.0
-c <float>	Coverage threshold	0.0
--cov-mode <int>	0=bidirectional, 1=target, 2=query	0
--exact-tmscore <int>	Use exact TM-score calculation	0

Examples:

```
# Rescore diagonals with structural alignment
foldseek structurerescorediagonal queryDB targetDB prefilterDB rescored_results

# TM-align based rescoring
foldseek structurerescorediagonal queryDB targetDB prefilterDB rescored_results
--alignment-type 1

# Filter by TM-score during rescoring
foldseek structurerescorediagonal queryDB targetDB prefilterDB rescored_results
--tmscore-threshold 0.5
```

5.2 Advanced Search Features

5.2.1 Iterative Search

Description: Perform iterative structural search for improved sensitivity.

Usage:

```
foldseek structuresearch queryDB targetDB iter1 tmp -s 9.5 --num-iterations 1
foldseek result2structprofile queryDB targetDB iter1 profileDB
foldseek structuresearch profileDB targetDB iter2 tmp --profile-search
```

5.2.2 Profile-Based Search

Description: Search using structural profiles for improved sensitivity.

Usage:


```
# Create structural profiles
foldseek result2structprofile queryDB targetDB search_results profileDB

# Search with profiles
foldseek structuresearch profileDB targetDB profile_results tmp --profile-search
```

5.2.3 GPU-Accelerated Search

Description: Use GPU acceleration for faster searches.

Usage:

```
# Prepare database for GPU
foldseek makepaddeddb targetDB targetDB_gpu

# GPU-accelerated search
foldseek structuresearch queryDB targetDB_gpu results tmp --gpu 1 --prefilter-mode 1
```

5.3 Search Output Formats

5.3.1 Tab-Separated Format (Default)

```
query target fidnt alnlen mismatch gapopen qstart qend tstart tend evalua bits
```

5.3.2 Custom Output with Structural Information

```
query target alntmscore qtmscore ttmscore lddt u t
```

5.3.3 SAM Format

```
@SQ SN:target1 LN:150
query1 0 target1 1 255 50M * 0 0 * * AS:i:100 NM:i:0
```

5.3.4 BLAST Format

```
# BLASTP 2.2.26+
# Query: query1
# Database: targetDB
query1 target1 95.67 150 6 2 1 150 1 150 0.0 300
```

5.4 Performance Optimization

5.4.1 Search Speed Optimization

- Use lower -s values for faster searches
- Enable GPU acceleration with --gpu 1 --prefilter-mode 1
- Use --max-seqs to limit results per query
- Use --max-rejected to stop early when no good hits are found

5.4.2 Search Sensitivity Optimization

- Use higher -s values for distant homology detection
- Enable --num-iterations for iterative refinement
- Use --alignment-type 1 for TM-align based scoring
- Adjust -e threshold based on requirements

5.4.3 Memory Optimization

- Use `--sort-by-structure-bits 0` to reduce memory usage
- Split large searches into smaller batches
- Use precomputed indexes for repeated searches
- Monitor memory usage with `-v 2`

These structure search modules provide comprehensive functionality for structural similarity analysis, from basic searches to advanced GPU-accelerated and profile-based methods.

6 Structure Clustering Modules

Foldseek provides specialized modules for clustering protein structures based on structural similarity, supporting both single-chain and multi-chain (multimer) clustering.

6.0.1 cluster

Description: Core clustering algorithm for structural data.

Usage:

```
foldseek cluster sequenceDB alignmentDB resultDB [options]
```

Parameters:

Parameter	Description	Default
<code>--cluster-mode <int></code>	0=set cover, 1=connected component, 2=greedy	0
<code>-c <float></code>	Coverage threshold	0.8
<code>--min-seq-id <float></code>	Minimum sequence identity	0.3
<code>--max-seqs <int></code>	Maximum sequences per query	1000
<code>--cluster-reassign <bool></code>	Reassign sequences after clustering	false

Examples:

```
# Set cover clustering (default)
foldseek cluster structuresDB alignments cluster_results

# Connected component clustering
foldseek cluster structuresDB alignments cluster_results --cluster-mode 1

# Greedy incremental clustering
foldseek cluster structuresDB alignments cluster_results --cluster-mode 2
```

6.0.2 clust

Description: Core clustering algorithm for structural data with advanced options.

Usage:

```
foldseek clust sequenceDB resultDB clusterDB [options]
```

Parameters:

Parameter	Description	Default
<code>--cluster-mode <int></code>	0=Set-Cover (greedy), 1=Connected component, 2=Greedy by length	0
<code>--max-iterations <int></code>	Maximum depth for connected component clustering	1000
<code>--similarity-type <int></code>	1=alignment score, 2=sequence identity	2
<code>--weights <string></code>	Weights for cluster prioritization	""
<code>--cluster-weight-threshold <float></code>	Weight threshold for cluster prioritization	0.9

Examples:

```
# Set cover clustering (default)
foldseek clust structuresDB alignments cluster_results

# Connected component clustering
foldseek clust structuresDB alignments cluster_results --cluster-mode 1

# Greedy clustering by length
foldseek clust structuresDB alignments cluster_results --cluster-mode 2

# Custom cluster weights
foldseek clust structuresDB alignments cluster_results --weights "0.5,0.3,0.2"
```

6.1 Clustering Algorithms

6.1.1 Set Cover Clustering (Mode 0)

Description: Approximates the NP-complete set cover problem using a greedy algorithm.

Algorithm: 1. Iteratively selects the structure with most connections 2. Forms cluster with all connected structures 3. Removes clustered structures from consideration 4. Repeats until all structures are clustered

Use Case: Optimal for finding representative structures with minimal redundancy.

6.1.2 Connected Component Clustering (Mode 1)

Description: Uses transitive connections to form larger clusters.

Algorithm: 1. Starts with structure having most connections 2. Performs breadth-first search to find all reachable structures 3. Forms cluster with all connected structures 4. Repeats with remaining structures

Use Case: Good for finding all related structures in a similarity graph.

6.1.3 Greedy Incremental Clustering (Mode 2)

Description: Similar to CD-HIT algorithm, sorts by length and clusters incrementally.

Algorithm: 1. Sorts structures by length (longest first) 2. For each structure, finds all similar structures 3. Forms cluster with representative and similar structures 4. Removes clustered structures from consideration

Use Case: Good for length-biased clustering where longer structures are preferred as representatives.

6.2 Clustering Output

6.2.1 Cluster Database Format

The clustering result database contains cluster assignments:

```
# Cluster 0 (representative: structure_0)
structure_0
structure_1
structure_2

# Cluster 1 (representative: structure_5)
structure_5
structure_3
structure_4
```

6.2.2 Tab-Separated Format

Convert to tab-separated format for analysis:

```
foldseek createtsv structuresDB structuresDB clusterDB cluster_results.tsv
```

Output format:

```
#cluster-representative cluster-member
structure_0 structure_0
structure_0 structure_1
structure_0 structure_2
structure_5 structure_5
structure_5 structure_3
structure_5 structure_4
```

6.2.3 Representative Sequences

Extract representative sequences:

```
foldseek createsubdb clusterDB structuresDB representativesDB
foldseek convert2fasta representativesDB representatives.fasta
```

6.2.4 All Member Sequences

Extract all sequences with cluster markers:

```
foldseek createseqfiledb structuresDB clusterDB all_membersDB
foldseek result2flat structuresDB structuresDB all_membersDB all_members.fasta
```

6.3 Advanced Clustering Features

6.3.1 Cascaded Clustering

Description: Multi-step clustering for improved sensitivity and speed.

Process: 1. **Step 1:** Fast clustering with low sensitivity to find initial clusters 2. **Step 2:** Cluster representatives from step 1 with higher sensitivity 3. **Step 3:** Final clustering with maximum sensitivity 4. **Merge:** Combine results from all steps

Usage:

```
foldseek structurecluster structuresDB cluster_results tmp --single-step-clustering false
```

6.3.2 Cluster Reassignment

Description: Reassigns sequences to better clusters after initial clustering.

Usage:

```
foldseek structurecluster structuresDB cluster_results tmp --cluster-reassign
```

6.3.3 Custom Clustering Criteria

Description: Use custom similarity thresholds for clustering.

Usage:

```
# High similarity clustering
foldseek structurecluster structuresDB cluster_results tmp \
  --tmscore-threshold 0.8 --lddt-threshold 0.9 -c 0.9

# Low similarity clustering (more clusters)
foldseek structurecluster structuresDB cluster_results tmp \
  --tmscore-threshold 0.3 --lddt-threshold 0.5 -c 0.5
```

6.4 Multimer Clustering

6.4.1 multimercluster

Description: Core multimer clustering functionality.

Usage:

```
foldseek multimercluster inputDB resultDB tmpDir [options]
```

Parameters:

Parameter	Description	Default
--multimer-tm-threshold <float>	Complex TM-score threshold	0.65
--chain-tm-threshold <float>	Individual chain TM-score threshold	0.5

Parameter	Description	Default
--interface-lddt-threshold <float>	Interface LDDT threshold	0.65
--cov-mode <int>	Coverage mode	0
--cluster-mode <int>	Clustering algorithm	0
-e <float>	E-value threshold	0.001

Examples:

```
# Basic multimer clustering
foldseek multimercluster complexesDB multimer_clusters tmp

# Strict multimer clustering
foldseek multimercluster complexesDB multimer_clusters tmp \
  --multimer-tm-threshold 0.8 --chain-tm-threshold 0.7 --interface-lddt-threshold
0.8
```

6.5 Clustering Evaluation

6.5.1 Cluster Quality Assessment

Description: Evaluate clustering quality using various metrics.

Usage:

```
# Calculate cluster statistics
foldseek result2stats structuresDB clusterDB cluster_stats

# Convert to TSV for analysis
foldseek createtsv structuresDB clusterDB cluster_stats cluster_stats.tsv
```

6.5.2 Cluster Size Distribution

Description: Analyze the distribution of cluster sizes.

Usage:

```
# Get cluster sizes
foldseek createtsv structuresDB structuresDB clusterDB cluster_sizes.tsv

# Analyze with standard tools
awk 'NR>1 {print $2}' cluster_sizes.tsv | sort | uniq -c | sort -nr
```

6.6 Performance Optimization

6.6.1 Speed Optimization

- Use --single-step-clustering for faster clustering
- Lower -s values for faster prefiltering
- Use --max-seqs to limit sequences per query
- Enable GPU acceleration when available

6.6.2 Memory Optimization

- Use `--sort-by-structure-bits 0` to reduce memory usage
- Split large datasets into smaller chunks
- Use precomputed indexes for repeated clustering
- Monitor memory usage with `-v 2`

6.6.3 Sensitivity Optimization

- Use higher `-s` values for distant homology detection
- Enable `--cluster-reassign` for better cluster assignments
- Use `--alignment-type 1` for TM-align based clustering
- Adjust coverage and identity thresholds based on requirements

6.7 Integration Examples

6.7.1 With Structure Prediction

```
# Predict structures
colabfold_batch sequences.fasta predicted_structures/

# Cluster predicted structures
foldseek structurecluster predicted_structures/ predicted_clusters tmp

# Extract representatives
foldseek createsubdb predicted_clusters predicted_structures/ representativesDB
foldseek convert2fasta representativesDB representatives.fasta
```

6.7.2 With Experimental Validation

```
# Cluster experimental structures
foldseek structurecluster experimental_structures/ experimental_clusters tmp

# Compare with predicted structures
foldseek structuresearch predicted_structures/ experimental_structures/
comparison_results tmp
```

6.7.3 Large-Scale Clustering

```
# Split large dataset
foldseek splitdb large_structuresDB large_structuresDB_split --split 10

# Cluster each split
for i in {0..9}; do
    foldseek structurecluster large_structuresDB_split_${i}_10
    cluster_results_${i} tmp_${i}
done

# Merge cluster results
foldseek mergeclusters large_structuresDB final_clusters cluster_results_*
```

These clustering modules provide comprehensive functionality for structural similarity-based grouping, supporting various algorithms and evaluation methods for different use cases.

7 Multimer Analysis Modules

Foldseek provides specialized modules for analyzing protein complexes (multimers), including search, clustering, and evaluation of multi-chain protein structures.

7.1 Core Multimer Modules

7.1.1 multimersearch

Description: Core multimer search functionality for protein complexes.

Usage:

```
foldseek multimersearch queryDB targetDB resultDB tmpDir [options]
```

Parameters:

Parameter	Description	Default
--multimer-tm-threshold <float>	Complex TM-score threshold	0.65
--chain-tm-threshold <float>	Individual chain TM-score threshold	0.5
--interface-lddt-threshold <float>	Interface LDDT threshold	0.65
--alignment-type <int>	0=3Di only, 1=TMalign, 2=3Di+AA	2
-e <float>	E-value threshold	0.001
--max-seqs <int>	Maximum sequences per query	1000
--comp-bias-corr <int>	Compositional bias correction	1
--mask <int>	Low complexity masking	1
--cov-mode <int>	0=bidirectional, 1=target, 2=query	0

Examples:

```
# Basic multimer search
foldseek multimersearch query_complexes.pdb targetDB multimer_results tmp

# Strict complex matching
foldseek multimersearch query_complexes.pdb targetDB multimer_results tmp \
  --multimer-tm-threshold 0.8 --chain-tm-threshold 0.6 --interface-lddt-threshold 0.7

# Custom alignment type
foldseek multimersearch query_complexes.pdb targetDB multimer_results tmp --
alignment-type 1
```

7.1.2 multimercluster

Description: Core multimer clustering functionality.

Usage:

`foldseek multimercluster inputDB resultDB tmpDir [options]`

Parameters:

Parameter	Description	Default
<code>--multimer-tm-threshold <float></code>	Complex TM-score threshold	0.65
<code>--chain-tm-threshold <float></code>	Individual chain TM-score threshold	0.5
<code>--interface-lddt-threshold <float></code>	Interface LDDT threshold	0.65
<code>--cov-mode <int></code>	Coverage mode	0
<code>--cluster-mode <int></code>	Clustering algorithm	0
<code>-e <float></code>	E-value threshold	0.001
<code>--max-seqs <int></code>	Maximum sequences per query	1000

Examples:

Basic multimer clustering

`foldseek multimercluster complexesDB multimer_clusters tmp`

Strict multimer clustering

`foldseek multimercluster complexesDB multimer_clusters tmp \`
`--multimer-tm-threshold 0.8 --chain-tm-threshold 0.7 --interface-lddt-threshold 0.8`

7.2 Multimer Scoring and Filtering

7.2.1 scoremultimer

Description: Score multimer alignments with detailed metrics.

Usage:

`foldseek scoremultimer queryDB targetDB alignmentDB scoreDB [options]`

Parameters:

Parameter	Description	Default
<code>--multimer-tm-threshold <float></code>	Complex TM-score threshold	0.0
<code>--chain-tm-threshold <float></code>	Chain TM-score threshold	0.0

Parameter	Description	Default
<code>--interface-lddt-threshold <float></code>	Interface LDDT threshold	0.0

Examples:

```
# Score multimer alignments
foldseek scoremultimer queryDB targetDB alignments scored_alignments

# Filter by complex score
foldseek scoremultimer queryDB targetDB alignments scored_alignments --multimer-tm-threshold 0.7
```

7.2.2 filtermultimer

Description: Filter multimer results based on complex-specific criteria.

Usage:

```
foldseek filtermultimer inputDB filterDB outputDB [options]
```

Parameters:

Parameter	Description	Default
<code>--multimer-tm-threshold <float></code>	Complex TM-score threshold	0.0
<code>--chain-tm-threshold <float></code>	Chain TM-score threshold	0.0
<code>--interface-lddt-threshold <float></code>	Interface LDDT threshold	0.0
<code>--filter-expression <string></code>	Custom filter expression	""

Examples:

```
# Filter by complex TM-score
foldseek filtermultimer alignments filtered_alignments outputDB --multimer-tm-threshold 0.7

# Filter by interface LDDT
foldseek filtermultimer alignments filtered_alignments outputDB --interface-lddt-threshold 0.8

# Custom filter expression
foldseek filtermultimer alignments filtered_alignments outputDB \
  --filter-expression '$3 > 0.7 && $5 > 0.8'
```

7.3 Multimer Result Processing

7.3.1 createmultimerreport

Description: Generate detailed reports for multimer alignments.

Usage:

```
foldseek createmultimerreport queryDB targetDB alignmentDB reportDB [options]
```

Parameters:

Parameter	Description	Default
--format-mode <int>	Output format	0
--db-output <bool>	Output as database	false

Examples:

```
# Create multimer report
foldseek createmultimerreport queryDB targetDB alignments multimer_report

# Convert to TSV
foldseek createtsv queryDB targetDB multimer_report multimer_report.tsv
```

7.3.2 expandmultimer

Description: Expand multimer search results with additional information.

Usage:

```
foldseek expandmultimer queryDB targetDB resultDB expandedDB [options]
```

Parameters:

Parameter	Description	Default
--max-seqs <int>	Maximum sequences per query	1000

Examples:

```
# Expand multimer results
foldseek expandmultimer queryDB targetDB results expanded_results

# Convert to detailed TSV
foldseek createtsv queryDB targetDB expanded_results expanded_results.tsv
```

7.4 Easy Multimer Workflows

7.4.1 easy-multimersearch

Description: User-friendly multimer search workflow.

Usage:

foldseek easy-multimersearch queryComplexes targetDB resultDB tmpDir [options]

Parameters:

Parameter	Description	Default
--multimer-tm-threshold <float>	Complex TM-score threshold	0.65
--chain-tm-threshold <float>	Chain TM-score threshold	0.5
--interface-lddt-threshold <float>	Interface LDDT threshold	0.65
--format-mode <int>	Output format	0
--format-output <string>	Custom output columns	query,target,fi- dent,alnlen,mis- match,gapopen,qstart,qend,tstart,tend,evalua- plexassignid

Examples:

```
# Basic easy multimer search
foldseek easy-multimersearch complex.pdb targetDB results tmp

# Custom output format
foldseek easy-multimersearch complex.pdb targetDB results tmp \
  --format-output "query,target,complexqtm_score,complexttm_score,complexassignid"

# HTML output
foldseek easy-multimersearch complex.pdb targetDB results.html tmp --format-mode
3
```

7.4.2 easy-multimercluster

Description: User-friendly multimer clustering workflow.

Usage:

foldseek easy-multimercluster inputComplexes resultDB tmpDir [options]

Parameters:

Parameter	Description	Default
--multimer-tm-threshold <float>	Complex TM-score threshold	0.65
--chain-tm-threshold <float>	Chain TM-score threshold	0.5
--interface-lddt-threshold <float>	Interface LDDT threshold	0.65

Parameter	Description	Default
--cov-mode <int>	Coverage mode	0
--cluster-mode <int>	Clustering algorithm	0

Examples:

```
# Basic multimer clustering
foldseek easy-multimercluster complexes/ multimer_clusters tmp

# Strict clustering criteria
foldseek easy-multimercluster complexes/ multimer_clusters tmp \
  --multimer-tm-threshold 0.8 --chain-tm-threshold 0.7 --interface-lddt-threshold
0.8
```

7.5 Multimer Output Formats

7.5.1 Tab-Separated Format (Default)

```
query target fidnt alnlen mismatch gapopen qstart qend tstart tend evalve bits
complexassignid
```

7.5.2 Custom Complex Output

```
query target complexqtm score complexttm score complexu complext complexassignid
```

7.5.3 Complex Report Format

```
query_complex target_complex matched_chains_q matched_chains_t qTM tTM U_matrix
T_vector assignment_id
```

7.5.4 Multimer Cluster Format

```
# Representative complex
representative_1
member_1
member_2
```

```
# Representative complex
representative_2
member_3
member_4
```

7.6 Multimer Analysis Concepts

7.6.1 Complex TM-Score

Description: TM-score calculated for the entire complex after optimal chain assignment.

Formula:

$$\text{complexTM} = \sum(\text{chainTM}_i * \text{chainLength}_i) / \text{totalComplexLength}$$

7.6.2 Chain Assignment

Description: Optimal assignment of query chains to target chains using Hungarian algorithm.

Algorithm: 1. Calculate similarity matrix between all query and target chains 2. Apply Hungarian algorithm for optimal assignment 3. Maximize total similarity score across all chains

7.6.3 Interface LDDT

Description: Local Distance Difference Test for protein-protein interfaces.

Calculation: - Identifies contacting residue pairs between chains - Calculates LDDT for interface residues - Provides measure of interface conservation

7.7 Advanced Multimer Features

7.7.1 Iterative Multimer Search

Description: Perform iterative search for improved multimer detection.

Usage:

```
# First iteration
foldseek multimersearch query_complexes.pdb targetDB iter1 tmp --multimer-tm-
threshold 0.8

# Create multimer profiles
foldseek result2structprofile query_complexes.pdb targetDB iter1
multimer_profiles

# Second iteration with profiles
foldseek multimersearch multimer_profiles targetDB iter2 tmp --profile-search
```

7.7.2 Multimer Quality Assessment

Description: Assess quality of multimer alignments.

Usage:

```
# Score multimer alignments
foldseek scoremultimer queryDB targetDB alignments scored_alignments

# Filter high-quality multimers
foldseek filtermultimer scored_alignments high_quality_multimers outputDB \
--multimer-tm-threshold 0.7 --interface-lddt-threshold 0.8
```

7.7.3 Multimer Visualization

Description: Generate visualizations of multimer alignments.

Usage:

```
# Create superimposed structures
foldseek convert2pdb queryDB targetDB multimer_alignments superimposed_complexes/

# Generate HTML report
```

```
foldseek createmultimerreport queryDB targetDB multimer_alignments
multimer_report
```

7.8 Performance Optimization

7.8.1 Speed Optimization

- Use appropriate thresholds to reduce search space
- Enable GPU acceleration when available
- Use `--max-seqs` to limit results per query
- Consider using precomputed indexes for large databases

7.8.2 Memory Optimization

- Use `--sort-by-structure-bits 0` to reduce memory usage
- Split large multimer searches into smaller batches
- Monitor memory usage with `-v 2`

7.8.3 Sensitivity Optimization

- Adjust TM-score and LDDT thresholds based on requirements
- Use higher thresholds for stricter matching
- Use lower thresholds for broader similarity detection

7.9 Integration Examples

7.9.1 With Structure Prediction

```
# Predict multimer structures
colabfold_batch --num-recycle 3 multimer_sequences.fasta predicted_multimers/

# Search predicted multimers
foldseek easy-multimersearch predicted_multimers/ reference_multimers/
multimer_results tmp
```

7.9.2 With Experimental Structures

```
# Compare predicted vs experimental multimers
foldseek easy-multimersearch predicted_multimers/ experimental_multimers/
comparison_results tmp

# Filter high-confidence matches
foldseek filtermultimer comparison_results high_confidence outputDB \
  --multimer-tm-threshold 0.8 --interface-lddt-threshold 0.7
```

7.9.3 Large-Scale Analysis

```
# Process large multimer datasets
for i in {1..100}; do
  foldseek easy-multimersearch batch_${i}.pdb large_multimer_db results_${i}
  tmp_${i}
done

# Merge and analyze results
```

```
foldseek mergedbs all_multimer_results results_* tmp
foldseek createmultimerreport queryDB targetDB all_multimer_results
multimer_analysis
```

These multimer modules provide comprehensive functionality for protein complex analysis, from basic search and clustering to advanced evaluation and visualization.

8 Structure Manipulation Modules

Foldseek provides various modules for processing, converting, and manipulating protein structure data, from database creation to format conversion and structural analysis.

8.1 Database Creation and Management

8.1.1 createdb

Description: Create a Foldseek database from protein structure files.

Usage:

```
foldseek createdb <i:directory>|.tsv>|<i:PDB|mmCIF|.gz>|tar|.gz>|DB> ... <i:PDB|
mmCIF|.gz>|tar|DB> <o:sequenceDB> [options]
```

Parameters:

Parameter	Description	Default
--prostt5-model <string>	Path to ProstT5 model for 3Di prediction	""
--gpu <int>	Enable GPU acceleration for ProstT5	0
--chain-name-mode <int>	Add chain to name: 0: auto, 1: always add	0
--write-lookup <int>	Write lookup file containing mapping from internal id, fasta id and file number	1
--coord-store-mode <int>	Coordinate storage mode: 1: C-alpha as float, 2: C-alpha as difference (uint16_t)	2
--write-mapping <int>	Write _mapping file containing mapping from internal id to taxonomic identifier	0
--input-format <int>	Format of input structures: 0: Auto-detect by extension, 1: PDB, 2: mmCIF, 3: mmJSON, 4: ChemComp, 5: Foldcomp	0

Parameter	Description	Default
<code>--file-include <string></code>	Include file names based on this regex	<code>“.*”</code>
<code>--file-exclude <string></code>	Exclude file names based on this regex	<code>“^\$”</code>

Examples:

```
# Process multiple files
foldseek createdb examples/1tim.pdb.gz examples/8tim.pdb.gz DB

# Process a directory containing PDB|mmCIF|.gz|tar|.gz|DB recursively, only one
# directory can be given
foldseek createdb examples/ DB

# Process a TSV file with a list of PDB|mmCIF|.gz|tar|.gz|DB, only one TSV can
# be given
foldseek createdb examples.tsv DB

# Process a directory or tar file and filter based on file name
# Note: --file-include and --file-exclude only apply to directory or tar input
foldseek createdb examples/ DB --file-include "pdb.gz$"

# Predict 3Di sequences from an amino acid FASTA file using ProstT5
foldseek databases ProstT5 weights tmp
foldseek createdb QUERY.fasta DB --prostt5-model weights

# Accelerate inference by one to two magnitudes using GPU(s) (--gpu 1)
foldseek createdb db.fasta db --prostt5-model weights --gpu 1
```

8.1.2 compressca

Description: Compress C α coordinate data.

Usage:

```
foldseek compressca <i:DB> <o:caDB> [options]
```

Parameters:

Parameter	Description	Default
<code>--coord-store-mode <int></code>	Coordinate storage mode: 1: C-alpha as float, 2: C-alpha as difference (uint16_t), 3: Plain text list of floats	2

Examples:

```
# Compress C $\alpha$  coordinates
foldseek compressca structDB structDB_compressed
```

8.2 Format Conversion Modules

8.2.1 convert2pdb

Description: Convert alignment results to superimposed PDB files.

Usage:

```
foldseek convert2pdb <i:Db> <o:pdbFile|pdbDir> [options]
```

Parameters:

Parameter	Description	Default
<code>--pdb-output-mode <int></code>	PDB output mode: 0: Single multi-model PDB file, 1: One PDB file per chain, 2: One PDB file per complex	0

Examples:

```
# Generate superimposed PDB files
foldseek convert2pdb queryDB superimposed_pdb/
```

8.2.2 convertalis

Description: Convert alignment results to various output formats.

Usage:

```
foldseek convertalis <i:queryDb> <i:targetDb> <i:alignmentDB> <o:alignmentFile>
[options]
```

Parameters:

Parameter	Description	Default
--format-mode <int>	Output format: 0: BLAST-TAB, 1: SAM, 2: BLAST-TAB + query/db length, 3: Pretty HTML, 4: BLAST-TAB + column headers, 5: Calpha only PDB super-posed to query	0
--format-output <string>	Choose comma separated list of output columns from: query,target,fi- query,target,evaluate,gapopen,pi- dent,fident,nident,qstart,qend,qlen,tstart,tend,tlen,alnlen,raw,bits,cigar,qseq,tseq,qh- match,qcov,tcov,qset,qsetid,tset,tsetid,taxid,tax- name,taxlineage,lddt,lddt- full,qca,tca,t,u,qtm-score,alntm-score,rmsd,prob,com- plexqtm-score,complexttm-	“query,target,fi- dent,alnlen,mis- match,gapopen,qstart,qend,tstart,tend,evaluate

Parameter	Description	Default
	score,complexu,complext,complex-assignid	
--exact-tmscore <int>	Turn on fast exact TMscore (slow), default is approximate	0
--gap-open <twin>	Gap open cost	"aa:10,nucl:10"
--gap-extend <twin>	Gap extension cost	"aa:1,nucl:1"
--threads <int>	Number of CPU-cores used (all by default)	10
--compressed <int>	Write compressed output	0
--db-output <bool>	Return a result DB instead of a text file	0

Examples:

```
# Create output in BLAST M8 format (12 columns):
# (1,2) identifiers for query and target sequences/profiles,
# (3) sequence identity, (4) alignment length, (5) number of mismatches,
# (6) number of gap openings, (7-8, 9-10) alignment start and end-position in
# query and in target,
# (11) E-value, and (12) bit score
foldseek convertalis queryDB targetDB result.m8

# Create a TSV containing pairwise alignments
foldseek convertalis queryDB targetDB result.tsv --format-output
query,target,qaln,taln

# Annotate a alignment result with taxonomy information from targetDB
foldseek convertalis queryDB targetDB result.tsv --format-output
query,target,taxid,taxname,taxlineage

# Create SAM output
foldseek convertalis queryDB targetDB result.sam --format-mode 1

# Create a TSV containing which query file a result comes from
foldseek createdb euk_queries.fasta bac_queries.fasta queryDB
foldseek convertalis queryDB targetDB result.tsv --format-output
qset,query,target
```

8.3 Structure Analysis Modules

8.3.1 aln2tmscore

Description: Convert alignment results to TM-scores.

Usage:

`foldseek aln2tmscore <i:queryDB> <i:targetDB> <i:alnDB> <o:resultDB> [options]`

Parameters:

Parameter	Description	Default
<code>--threads <int></code>	Number of CPU-cores used (all by default)	10
<code>--compressed <int></code>	Write compressed output	0

Examples:

```
# Convert alignments to TM-scores
foldseek aln2tmscore queryDB targetDB alignments tm_scores
```

8.3.2 result2profile

Description: Convert search results to sequence profiles.

Usage:

`foldseek result2profile <i:queryDB> <i:targetDB> <i:resultDB> <o:profileDB> [options]`

Parameters:

Parameter	Description	Default
<code>--comp-bias-corr <int></code>	Correct for locally biased amino acid composition (range 0-1)	1
<code>--comp-bias-corr-scale <float></code>	Correct for locally biased amino acid composition (range 0-1)	1.000
<code>-e <double></code>	List matches below this E-value (range 0.0-inf)	10.0
<code>--gap-open <twin></code>	Gap open cost	"aa:10,nucl:10"
<code>--gap-extend <twin></code>	Gap extension cost	"aa:1,nucl:1"
<code>--mask-profile <int></code>	Mask query sequence of profile using tantan	1
<code>--e-profile <double></code>	Include sequences matches with < E-value thr. into the profile (≥ 0.0)	0.001
<code>--wg <bool></code>	Use global sequence weighting for profile calculation	0
<code>--filter-msa <int></code>	Filter msa: 0: do not filter, 1: filter	1
<code>--filter-min-enable <int></code>	Only filter MSAs with more than N sequences, 0 always filters	0

Parameter	Description	Default
<code>--max-seq-id <float></code>	Reduce redundancy of output MSA using max. pairwise sequence identity	0.900
<code>--qid <string></code>	Reduce diversity of output MSAs using min.seq. identity with query sequences	"0.0"
<code>--qsc <float></code>	Reduce diversity of output MSAs using min. score per aligned residue with query sequences	-20.000
<code>--cov <float></code>	Filter output MSAs using min. fraction of query residues covered by matched sequences	0.000
<code>--diff <int></code>	Filter MSAs by selecting most diverse set of sequences, keeping at least this many seqs in each MSA block of length 50	1000
<code>--pseudo-cnt-mode <int></code>	Use 0: substitution-matrix or 1: context-specific pseudocounts	0
<code>--profile-output-mode <int></code>	Profile output mode: 0: binary log-odds 1: human-readable frequencies	0
<code>--allow-deletion <bool></code>	Allow deletions in a MSA	0
<code>--threads <int></code>	Number of CPU-cores used (all by default)	10
<code>--compressed <int></code>	Write compressed output	0

Examples:

```
# Convert search results to profiles
foldseek result2profile queryDB targetDB search_results profileDB

# Custom profile filtering
foldseek result2profile queryDB targetDB search_results profileDB --max-seq-id
0.8 --qid 0.3 --cov 0.5
```

8.4 Structure Processing Workflows

8.4.1 Structure Database Creation Pipeline

```
# 1. Create basic database
foldseek createdb structures/ structDB
```

```
# 2. Create index for fast search
foldseek createindex structDB tmp
```

```
# 3. Compress coordinates
foldseek compressca structDB structDB_compressed
```

8.4.2 Structure Conversion Pipeline

```
# 1. Perform structural search
foldseek easy-search query.pdb structDB search_results tmp
```

```
# 2. Convert to superimposed PDBs
foldseek convert2pdb queryDB superimposed_structures/
```

```
# 3. Convert to various formats
foldseek convertalis queryDB structDB search_results results.tsv --format-output
query,target,fident,alnlen,evalue,bits
```

```
# 4. Create profiles from results
foldseek result2profile queryDB structDB search_results struct_profiles
```

8.4.3 Quality Assessment Pipeline

```
# 1. Perform alignment
foldseek structurealign queryDB targetDB alignments
```

```
# 2. Convert to TM-scores
foldseek aln2tmscore queryDB targetDB alignments tm_scores
```

```
# 3. Generate quality report
foldseek convertalis queryDB targetDB tm_scores quality_report.tsv --format-
output query,target,alntmscore,qtmscore,ttmscore
```

8.5 Advanced Structure Manipulation

8.5.1 Custom Structure Filtering

```
# 1. Create alignment database
foldseek structurealign queryDB targetDB alignments

# 2. Filter by TM-score
foldseek filterdb alignments filtered_alignments --filter-expression '$3 > 0.7'

# 3. Convert to readable format
foldseek convertalis queryDB targetDB filtered_alignments filtered_results.tsv
```

8.5.2 Structure Profile Creation

```
# 1. Perform search
foldseek easy-search query.pdb structDB search_results tmp

# 2. Create profiles from results
```

```
foldseek result2profile queryDB structDB search_results struct_profiles
```

```
# 3. Search with profiles
```

```
foldseek search struct_profiles structDB profile_search_results tmp
```

8.5.3 Batch Structure Processing

```
# Process multiple structure files
```

```
for i in {1..100}; do
    foldseek createdb batch_${i}/ batch_${i}_db
    foldseek easy-search batch_${i}_db reference_db batch_${i}_results tmp_${i}
done
```

```
# Merge all results
```

```
foldseek mergedbs all_results batch_*_results tmp
```

8.6 Performance Optimization

8.6.1 Database Creation Optimization

- Use `--chain-name-mode 0` for automatic chain naming
- Use `--coord-store-mode 2` for efficient coordinate storage
- Use `--write-lookup 1` to enable fast lookups
- Use GPU acceleration for ProstT5 when available

8.6.2 Memory Optimization

- Use `--sort-by-structure-bits 0` to reduce memory usage
- Process large datasets in batches
- Use compressed databases when possible
- Monitor memory usage with `-v 2`

8.6.3 Speed Optimization

- Use precomputed indexes for repeated operations
- Enable GPU acceleration for 3Di prediction
- Use appropriate compression levels
- Split large operations into smaller chunks

8.7 Integration Examples

8.7.1 With Structure Prediction

```
# Predict structures with AlphaFold
```

```
alphafold --fasta sequences.fasta --output structures/
```

```
# Create Foldseek database
```

```
foldseek createdb structures/ predicted_structDB
```

```
# Search against reference
```

```
foldseek easy-search predicted_structDB reference_structDB validation_results tmp
```

8.7.2 With Experimental Validation

```
# Convert experimental structures to database
foldseek createdb experimental_structures/ exp_structDB

# Compare predicted vs experimental
foldseek structurealign predicted_structDB exp_structDB comparison_results

# Generate quality metrics
foldseek aln2tmscore predicted_structDB exp_structDB comparison_results
quality_metrics
```

8.7.3 Large-Scale Processing

```
# Process large structure datasets
foldseek splitdb large_structDB large_structDB_split --split 100

# Process each split
for i in {0..99}; do
    foldseek createdb large_structDB_split_${i}_100 large_split_${i}_db
    foldseek easy-search large_split_${i}_db reference_db split_${i}_results
    tmp_${i}
done

# Merge results
foldseek mergedbs all_struct_results split_*_results tmp
```

These structure manipulation modules provide comprehensive functionality for processing, converting, and analyzing protein structure data at scale.

9 Database Management Modules

Foldseek provides comprehensive database management capabilities, from downloading pre-built databases to creating custom indexes and managing large-scale structural data collections.

9.1 Database Download and Setup

9.1.1 databases

Description: Download and set up pre-built structural databases.

Usage:

```
foldseek databases <name> <o:sequenceDB> <tmpDir> [options]
```

Available Databases:

Database	Type	Taxonomy	URL
Alphafold/UniProt	Aminoacid	yes	https://alphafold.ebi.ac.uk/
Alphafold/UniProt50-minimal	Aminoacid	yes	https://alphafold.ebi.ac.uk/

Database	Type	Taxonomy	URL
Alphafold/UniProt50	Aminoacid	yes	https://alphafold.ebi.ac.uk/
Alphafold/Proteome	Aminoacid	yes	https://alphafold.ebi.ac.uk/
Alphafold/Swiss-Prot	Aminoacid	yes	https://alphafold.ebi.ac.uk/
ESMAAtlas30	Aminoacid	-	https://esmatlas.com
PDB	Aminoacid	yes	https://www.rcsb.org
CATH50	Aminoacid	yes	https://www.cath.info
BFMD	Aminoacid	yes	https://foldseek.steineggerlab.workers.dev/bfmd.version
BFVD	Aminoacid	yes	https://bfvd.steineggerlab.workers.dev
ProstT5	Aminoacid	-	https://huggingface.co/Rostlab/ProstT5

Parameters:

Parameter	Description	Default
<code>--tsv <bool></code>	Return output in TSV format	0
<code>--force-reuse <bool></code>	Reuse tmp files in tmp/latest folder ignoring parameters and version changes	0
<code>--remove-tmp-files <bool></code>	Delete temporary files	0
<code>--compressed <int></code>	Write compressed output	0
<code>--threads <int></code>	Number of CPU-cores used (all by default)	10

Examples:

```
# Download AlphaFold database for Swiss-Prot
foldseek databases Alphafold/Swiss-Prot afdB_swissprot tmp

# Download PDB database
foldseek databases PDB pdb tmp

# Download with multiple threads
foldseek databases Alphafold/UniProt afdB_uniprot tmp --threads 8

# Get database list in TSV format
foldseek databases --tsv
```

9.2 Index Creation and Management

9.2.1 createindex

Description: Create precomputed index for faster structural searches.

Usage:

`foldseek createindex <i:sequenceDB> <tmpDir> [options]`

Parameters:

Parameter	Description	Default
<code>--index-subset <int></code>	Create specialized index with subset of entries: 0: normal index, 1: index without headers, 2: index without prefiltering data, 4: index without aln (for cluster db), Flags can be combined bit wise	0
<code>--check-compatible <int></code>	0: Always recreate index, 1: Check if recreating index is needed, 2: Fail if index is incompatible	0
<code>--split <int></code>	Split input into N equally distributed chunks. 0: set the best split automatically	0
<code>--split-memory-limit <byte></code>	Set max memory per split. E.g. 800B, 5K, 10M, 1G. Default (0) to all available system memory	0
<code>--kmer-size <int></code>	k-mer length (0: automatically set to optimum)	0
<code>--seed-sub-mat <twin></code>	Substitution matrix file for k-mer generation	"aa:3di.out,nuc:3di.out"
<code>--comp-bias-corr <int></code>	Correct for locally biased amino acid composition (range 0-1)	1
<code>--comp-bias-corr-scale <float></code>	Correct for locally biased amino acid composition (range 0-1)	1.000
<code>--max-seqs <int></code>	Maximum results per query sequence allowed to pass the prefilter (affects sensitivity)	1000
<code>--mask <int></code>	Mask sequences in prefilter stage with tantan: 0: w/o low complexity	0

Parameter	Description	Default
	masking, 1: with low complexity masking	
--mask-prob <float>	Mask sequences is probablity is above threshold	1.000
--mask-lower-case <int>	Lowercase letters will be excluded from k-mer search 0: include region, 1: exclude region	1
--mask-n-repeat <int>	Repeat letters that occure > threshold in a rwo	6
--spaced-kmer-mode <int>	0: use consecutive positions in k-mers; 1: use spaced k-mers	1
--spaced-kmer-pattern <string>	User-specified spaced k-mer pattern	""
-s <float>	Sensitivity: 1.0 faster; 4.0 fast; 7.5 sensitive	9.500
--k-score <twinn>	k-mer threshold for generating similar k-mer lists	"seq:2147483647,prof:2147483647"
--threads <int>	Number of CPU-cores used (all by default)	10
--compressed <int>	Write compressed output	0
--remove-tmp-files <bool>	Delete temporary files	1

Examples:

```
# Create protein sequence index
foldseek createindex sequenceDB tmp

# Create specialized index with subset of entries
foldseek createindex targetDB tmp --index-subset 2

# Create index with custom parameters
foldseek createindex targetDB tmp --kmer-size 6 --spaced-kmer-mode 1
```

9.2.2 createclusearchdb

Description: Create searchable databases from clustered results.

Usage:

```
foldseek createclusearchdb <i:sequenceDB> <i:clusterDB> <o:sequenceDB> [options]
```

Parameters:

Parameter	Description	Default
<code>--db-suffix-list <string></code>	Suffixes for database to be split in rep/seq	"_h,_ss,_ca"
<code>--threads <int></code>	Number of CPU-cores used (all by default)	10
<code>--compressed <int></code>	Write compressed output	0

Examples:

```
# Cluster database and build a searchable db
foldseek cluster sequenceDB clusterDB tmp --min-seq-id 0.3
foldseek createclusearchdb sequenceDB clusterDB clusterSearchDb
foldseek search sequenceDB clusterSearchDb aln tmp --cluster-search 1
```

9.3 Database Operations

9.3.1 Database Information and Utilities

Database information can be obtained using standard system tools:

```
# List database files
ls -la structDB*

# Check database size
du -sh structDB*

# Verify database integrity
foldseek createindex structDB tmp --check-compatible 1
```

9.4 Custom Database Creation

9.4.1 From Structure Files

```
# 1. Create basic database
foldseek createdb structures/ customDB

# 2. Create index
foldseek createindex customDB tmp
```

9.4.2 From Sequence Files with ProST5

```
# 1. Download ProST5 model
foldseek databases ProST5 weights tmp

# 2. Create database with 3Di prediction
foldseek createdb sequences.fasta structDB --prostt5-model weights/

# 3. GPU-accelerated creation
foldseek createdb sequences.fasta structDB --prostt5-model weights/ --gpu 1
```

```
# 4. Create index
foldseek createindex structDB tmp
```

9.4.3 From AlphaFold Results

```
# 1. Organize AlphaFold results
mkdir alphafold_results
# Copy AlphaFold PDB files to alphafold_results/
```

```
# 2. Create database
foldseek createdb alphafold_results/ afdb
```

```
# 3. Create index
foldseek createindex afdb tmp
```

9.5 Database Maintenance

9.5.1 Database Validation

```
# Check database integrity
foldseek createindex structDB tmp --check-compatible 1

# Validate against original files
foldseek convertalis structDB structDB structDB validation_results --format-
output query
```

9.5.2 Database Cleanup

```
# Remove temporary files
rm -f structDB.tmp*

# Compress database
foldseek createdb structDB structDB_compressed --compressed 1

# Remove original
rm -rf structDB*

# Rename compressed version
mv structDB_compressed* structDB
```

9.5.3 Database Backup

```
# Create compressed backup
tar -czf structDB_backup.tar.gz structDB*

# Create index backup
tar -czf structDB_index_backup.tar.gz structDB.idx*

# Verify backup
foldseek createindex structDB tmp --check-compatible 1
```

9.6 Performance Optimization

9.6.1 Index Optimization

```
# Create optimal index for structural search
foldseek createindex structDB tmp --index-subset 2

# Create optimal index for sequence search
foldseek createindex structDB tmp --index-subset 1

# Create comprehensive index
foldseek createindex structDB tmp --index-subset 0
```

9.6.2 Memory Optimization

```
# Use memory-efficient indexing
foldseek createindex structDB tmp --split 8

# Use compressed databases
foldseek createdb structDB structDB_compressed --compressed 1
```

9.6.3 GPU Optimization

```
# Use GPU acceleration for database creation
foldseek createdb sequences.fasta structDB --prott5-model weights/ --gpu 1

# Create index for GPU-optimized database
foldseek createindex structDB tmp
```

9.7 Large-Scale Database Management

9.7.1 Batch Processing

```
# Process large datasets in batches
for i in {1..100}; do
    foldseek createdb batch_${i}/ batch_${i}_db
    foldseek createindex batch_${i}_db tmp_${i}
done

# Create cluster search database
foldseek cluster largeDB clusterDB tmp
foldseek createclusearchdb largeDB clusterDB largeDB_search

# Create final index
foldseek createindex largeDB_search tmp
```

9.7.2 Distributed Processing

```
# Process large datasets in parallel
for i in {1..100}; do
    foldseek createdb batch_${i}/ batch_${i}_db
    foldseek createindex batch_${i}_db tmp_${i} &
done
wait
```

```
# Create cluster search database
foldseek cluster largeDB clusterDB tmp
foldseek createclusearchdb largeDB clusterDB largeDB_search
```

9.7.3 Creating Searchable Databases from New Structures

```
# Create new database with additional structures
foldseek createdb new_structures/ newDB

# Create cluster search database for efficient searching
foldseek cluster newDB clusterDB tmp
foldseek createclusearchdb newDB clusterDB newDB_search

# Create index for fast search operations
foldseek createindex newDB_search tmp
```

9.8 Integration Examples

9.8.1 With High-Performance Computing

```
# Use MPI for large database operations
mpirun -np 64 foldseek createindex largeDB tmp

# Use job scheduler
sbatch --nodes=4 --ntasks=128 create_index_job.sh
```

9.8.2 With Cloud Storage

```
# Download from cloud storage
aws s3 cp s3://bucket/structures/ ./structures/ --recursive

# Create database
foldseek createdb structures/ structDB

# Upload to cloud storage
aws s3 cp structDB* s3://bucket/databases/ --recursive
```

9.8.3 With Version Control

```
# Create versioned database
foldseek createdb structures/ structDB_v1.0

# Create index
foldseek createindex structDB_v1.0 tmp

# Tag version
git tag -a v1.0 -m "Initial database version"

# Create backup
tar -czf structDB_v1.0_backup.tar.gz structDB_v1.0*
```

These database management modules provide comprehensive functionality for handling large-scale structural databases, from initial creation and indexing to maintenance and optimization.

9.9 Database Format Comparison with MMseqs2

9.9.1 Structural vs Sequence Databases

Feature	MMseqs2 Database	Foldseek Database	Key Difference
Data Types	Amino acid sequences Nucleotide sequences Profile databases	3Di sequences Amino acid sequences C α coordinates Secondary structure	Foldseek adds structural data
File Components	database, database.index database_h, database_h.index database.lookup, database.dbtype	database, database.index database_h, database_h.index database_ca, database_ss database.lookup, database.dbtype	Additional structural files
Memory Usage	~7 bytes per residue (index) ~1 byte per residue (sequences)	~1 byte per residue (3Di) ~8 bytes per residue (C α coords) ~1 byte per residue (sequences)	More compact for structural data
Index Structure	K-mer index for sequences	3Di k-mer index Structural descriptors	Optimized for structural search

9.9.2 Database Creation Process

9.9.2.1 MMseqs2 Database Creation

```
# Convert FASTA sequences to MMseqs2 database
mmseqs createdb sequences.fasta seqDB
```

```
# Files created:
# - seqDB (amino acid sequences)
# - seqDB.index (sequence index)
# - seqDB_h (FASTA headers)
# - seqDB_h.index (header index)
# - seqDB.lookup (ID mapping)
# - seqDB.dbtype (database type)
```

9.9.2.2 Foldseek Database Creation

```
# Convert structures to Foldseek database
foldseek createdb structures/ structDB
```



```
# Files created:
# - structDB (3Di + AA sequences)
# - structDB.index (structural index)
# - structDB_h (structure headers)
# - structDB_h.index (header index)
# - structDB_ca (Cα coordinates)
# - structDB_ss (secondary structure)
# - structDB.lookup (ID mapping)
# - structDB.dbtype (database type)
```

9.9.3 Index Structure Differences

9.9.3.1 MMseqs2 Index

- **K-mer size:** Variable (6-14 amino acids)
- **Alphabet:** 20 amino acids (or reduced alphabet)
- **Memory usage:** ~7 bytes per residue
- **Search type:** Sequence similarity
- **Substitution matrix:** BLOSUM62, VTML matrices

9.9.3.2 Foldseek Index

- **K-mer size:** Variable (3Di alphabet)
- **Alphabet:** 3-letter 3Di (H/E/C)
- **Memory usage:** ~1 byte per residue
- **Search type:** Structural similarity
- **Substitution matrix:** 3Di-specific matrix
- **Additional data:** Structural descriptors, Cα coordinates

9.9.4 Database Type Specifications

Database Type	MMseqs2	Foldseek	Description
Amino Acid	0	0	Standard protein sequences
Nucleotide	1	1	DNA/RNA sequences
Profiles	2	2	Profile databases
Structural	N/A	3	3Di + structural data
Alignment Results	5	5	Search results
Clustering Results	6	6	Cluster assignments
Prefiltering Results	7	7	Prefilter hits
Taxonomy Results	8	8	Taxonomic assignments

9.9.5 Performance Implications

9.9.5.1 Memory Efficiency

- **MMseqs2**: Requires ~7 bytes per residue for k-mer indexing
- **Foldseek**: Requires ~1 byte per residue for 3Di indexing
- **Advantage**: Foldseek uses ~7x less memory for indexing

9.9.5.2 I/O Efficiency

- **MMseqs2**: Memory-maps sequence data and k-mer indices
- **Foldseek**: Memory-maps 3Di data, C α coordinates, and structural indices
- **Advantage**: Foldseek's smaller indices enable faster loading

9.9.5.3 Search Speed

- **MMseqs2**: Optimized for sequence similarity search
- **Foldseek**: Optimized for structural similarity search
- **Trade-off**: Different optimization targets for different data types

9.9.6 Compatibility and Integration

9.9.6.1 Shared Database Operations

Both tools support: - Memory-mapped I/O for efficient access - Compressed database storage - Index creation and management - Database splitting for large datasets - Sub-database creation

9.9.6.2 Unique Foldseek Operations

- 3Di sequence generation from structures
- C α coordinate extraction and storage
- Secondary structure assignment
- Structural index creation
- GPU-optimized database creation

9.9.6.3 Cross-Compatibility

- Foldseek can read MMseqs2 sequence databases
- MMseqs2 cannot read Foldseek structural databases
- Both use compatible result formats
- Shared taxonomy and clustering modules

10 Foldseek Expert Manual: Advanced Database Interactions and Structural Alignment

This document provides a detailed look into how various Foldseek modules interact with structural databases and perform advanced structural alignments. Understanding these interactions is key to building custom structural analysis workflows and troubleshooting complex pipelines.

10.1 Core Concepts

This expert manual focuses on advanced usage and database/alignment behavior. For internals and formats, see: - Developer Manual ([foldseek_docs/developer_manual.md](#)) for architecture,

3Di internals, and database file layout - Databases (foldseek_docs/submodules/databases.md) for user-facing database operations and formats

10.1.1 Header Database Handling Patterns

When Header and Main Databases are Processed Together:

1. Database Creation (createdb):

- Creates header and sequence databases simultaneously
- Both use identical numeric IDs for corresponding entries
- Headers stored in database_h and database_h.index

2. Format Conversion Operations:

- convertalis: Reads both sequence and header databases for output formatting
- convert2fasta: Combines header and sequence data for FASTA output

When Header and Main Databases are Processed Separately:

1. Core Search Operations:

- structuresearch: Only reads sequence data for k-mer matching
- structurealign: Only reads sequence and coordinate data for alignment
- multimersearch: Only reads structural data for complex analysis

2. Index-Only Operations:

- Operations that only need structural indices for prefiltering
- GPU-accelerated searches that stream data from disk

10.1.2 Performance Optimization Patterns

Memory-Efficient Access:

```
// Load only what you need
DBReader<unsigned int> reader(dbPath, indexPath, threads,
                             DBReader<unsigned int>::USE_INDEX); // Index only
DBReader<unsigned int> caReader(caPath, caIndexPath, threads,
                               DBReader<unsigned int>::USE_DATA); // Data only
```

Streaming Large Databases:

```
// Process in chunks for memory efficiency
for (size_t chunkStart = 0; chunkStart < totalEntries; chunkStart += chunkSize) {
    size_t chunkEnd = std::min(chunkStart + chunkSize, totalEntries);
    // Process chunk
    for (size_t i = chunkStart; i < chunkEnd; ++i) {
        const char* data = reader.getData(i, thread_idx);
        // Process data
    }
}
```

Parallel Database Access:

```
// Multiple threads accessing different database segments
#pragma omp parallel for
for (size_t i = 0; i < numQueries; ++i) {
```

```

    int thread_idx = omp_get_thread_num();
    const char* queryData = queryReader.getData(i, thread_idx);
    // Process query independently
}

```

10.2 Module Documentation

10.2.1 createdb

Module Type: Database Creation

Function: Converts protein structure files (PDB, mmCIF, mmJSON) into Foldseek databases.

Database Interactions:

- **Input Reading:**
 - Reads structural data from PDB/mmCIF/mmJSON files
 - Parses atomic coordinates, chain information, and metadata
 - Supports compressed files (.gz, .bz2) and piped input
- **Output Writing:**
 - **database:** Concatenated 3Di sequences, amino acid sequences, and descriptors
 - **database_ca:** C α coordinates for each residue
 - **database_ss:** Secondary structure assignments
 - **database_h:** Structure headers and metadata
 - **database.lookup:** Maps internal IDs to original structure identifiers
 - **database.dbtype:** Binary type specification
- **3Di Generation:**
 - **Secondary Structure Assignment:** Uses DSSP algorithm to assign H/E/C states
 - **Coordinate Processing:** Extracts C α coordinates for TM-score calculations
 - **Chain Handling:** Supports multi-chain structures with configurable naming
 - **ProstT5 Integration:** Optional 3Di prediction from sequences using language models

10.2.2 structuresearch

Module Type: Structural Search

Function: Performs fast structural similarity search using 3Di representation.

Database Interactions:

- **Input Reading:**
 - **Query Database (<i:queryDB>):** Reads 3Di sequences and structural descriptors
 - **Target Database (<i:targetDB>):** Reads target 3Di sequences for indexing
 - **Index Files:** Uses precomputed k-mer indices for fast lookup
- **Output Writing:**
 - **Result Database (<o:resultDB>):** Structural alignment results
 - Each entry contains: target ID, structural score, TM-score, LDDT, E-value, coordinates
- **Structural Scoring:**
 - **3Di Alignment:** Gotoh-Smith-Waterman with 3Di substitution matrix

- **TM-score Calculation:** Global structural alignment using TM-align algorithm
- **LDDT Scoring:** Local structural conservation assessment
- **E-value Computation:** Statistical significance using extreme value distribution

10.2.3 **structurealign**

Module Type: Structural Alignment

Function: Performs detailed structural alignments between query and target structures.

Database Interactions:

- **Input Reading:**
 - **Query Database (<i:queryDB>):** Reads 3Di sequences and C α coordinates
 - **Target Database (<i:targetDB>):** Reads target 3Di sequences and C α coordinates
 - **Prefiltering Results:** Uses precomputed candidate pairs
- **Output Writing:**
 - **Alignment Database (<o:alignmentDB>):** Detailed structural alignments
 - Contains: alignment paths, TM-scores, rotation matrices, translation vectors
- **Alignment Algorithms:**
 - **3Di+AA Alignment:** Combines 3Di structural and amino acid sequence information
 - **TM-align:** Global structural alignment with TM-score optimization
 - **SIMD Optimization:** Vectorized Smith-Waterman for 3Di sequences

10.2.4 **multimersearch**

Module Type: Multimer Search

Function: Searches protein complexes (multimers) against target databases.

Database Interactions:

- **Input Reading:**
 - **Query Complexes (<i:queryDB>):** Reads multi-chain structures
 - **Target Complexes (<i:targetDB>):** Reads target multi-chain structures
 - **Chain Information:** Parses chain IDs and interfaces
- **Output Writing:**
 - **Complex Alignment Database (<o:alignmentDB>):** Complex-level alignments
 - Contains: complex TM-scores, chain assignments, interface LDDT scores
- **Complex Scoring:**
 - **Chain Matching:** Optimal assignment of query chains to target chains
 - **Interface Analysis:** LDDT scoring of chain-chain interfaces
 - **Complex TM-score:** Weighted combination of individual chain TM-scores
 - **Assignment Optimization:** Hungarian algorithm for optimal chain matching

10.2.5 **talign**

Module Type: TM-align Integration

Function: Performs TM-align based structural alignments.

Database Interactions:

- **Input Reading:**
 - **Structure Databases:** Reads C α coordinates and sequences
 - **Alignment Parameters:** Uses precomputed structural similarities
- **Output Writing:**
 - **TM-align Results:** TM-scores, rotation matrices, translation vectors
 - Contains: global alignment scores and superposition parameters
- **TM-score Implementation:**
 - **Rotation Matrix:** Optimal rotation using quaternion method
 - **Translation Vector:** Optimal translation for superposition
 - **Score Normalization:** TM-score normalized by protein length
 - **Statistical Significance:** P-value calculation using empirical distributions

10.2.6 convert2pdb

Module Type: Format Conversion

Function: Converts structural alignment results to superimposed PDB files.

Database Interactions:

- **Input Reading:**
 - **Query Database (<i:queryDB>):** Reads query structures and coordinates
 - **Target Database (<i:targetDB>):** Reads target structures and coordinates
 - **Alignment Database (<i:alignmentDB>):** Reads alignment transformations
- **Output Writing:**
 - **Superimposed PDB Files:** One PDB file per alignment
 - Each file contains: superimposed C α atoms, transformation metadata
- **Coordinate Transformation:**
 - **Matrix Application:** Applies rotation and translation matrices
 - **Chain Preservation:** Maintains original chain IDs and metadata
 - **Format Compliance:** Generates valid PDB format with proper headers

10.3 Advanced Database Operations

10.3.1 Index Creation and Management

10.3.1.1 Structural Index Architecture

Foldseek creates specialized indices optimized for different types of structural queries:

Index Types: - **Subset 0 (Comprehensive):** All data types (sequences, coordinates, secondary structure) - **Subset 1 (Sequence-only):** Smaller, faster loading for sequence-based operations - **Subset 2 (Structure-only):** Optimized for structural searches and alignments

```
# Create comprehensive structural index
foldseek createindex targetDB tmp --index-subset 0

# Create sequence-only index (smaller, faster loading)
```

```
foldseek createindex targetDB tmp --index-subset 1

# Create structure-only index (for structural searches)
foldseek createindex targetDB tmp --index-subset 2
```

10.3.1.2 Index File Structure

The index creation process generates several coordinated files:

```
targetDB.index          # Main index file
targetDB.index.dbtype   # Index database type
targetDB.index.lookup   # Index lookup table
targetDB.index_h        # Header index
targetDB.index_h.index  # Header index index
```

10.3.1.3 GPU-Optimized Databases

Padded Database Creation:

```
# Create padded database for GPU searches
foldseek makepaddeddb targetDB targetDB_gpu

# Use with GPU acceleration
foldseek search queryDB targetDB_gpu resultDB tmp --gpu 1
```

GPU Database Structure: - **Padding:** Sequences padded to fixed lengths for efficient GPU processing - **Memory Layout:** Optimized for coalesced memory access patterns - **Streaming:** Supports databases larger than GPU memory through streaming

10.3.1.4 Database Splitting for Large-Scale Operations

Memory-Efficient Processing:

```
# Split large database for memory management
foldseek splitdb largeDB largeDB_split --split 10

# Process each split separately
for i in {0..9}; do
    foldseek search queryDB largeDB_split_${i}_10 result_${i} tmp_${i}
done

# Merge results
foldseek mergedbs resultDB result_0 result_1 result_2 result_3 result_4 result_5
result_6 result_7 result_8 result_9
```

Split Database Structure:

```
largeDB_split_0_10/    # First split (entries 0-9)
largeDB_split_1_10/    # Second split (entries 10-19)
...
largeDB_split_9_10/    # Last split (entries 90-99)
```

10.3.1.5 Custom Database Creation

From Structure Prediction Results:

```
# Convert AlphaFold results to Foldseek database
```

```
foldseek createdb alphafold_results/ afdb
```

```
# Add taxonomy information
```

```
foldseek createtaxdb afdb tmp --tax-mapping-file uniprot_taxid.tsv
```

From Experimental Structures:

```
# Process PDB structures with custom chain handling
```

```
foldseek createdb pdb_structures/ custom_db --chain-name-mode 1
```

```
# Create index optimized for structural searches
```

```
foldseek createindex custom_db tmp --index-subset 2
```

10.3.1.6 Database Concatenation

Combining Multiple Databases:

```
# Concatenate multiple structural databases
```

```
foldseek concatdbs db1 db2 combined_db
```

```
# Note: Header databases must be concatenated separately
```

```
foldseek concatdbs db1_h db2_h combined_db_h
```

Concatenation Process: 1. **Key Renumbering:** Assigns new sequential IDs to entries from second database 2. **Index Merging:** Combines index files from both databases 3. **Lookup Table Updates:** Updates lookup tables to maintain ID mapping 4. **Header Integration:** Merges header databases separately

10.3.2 Database Performance Optimization

10.3.2.1 Memory Usage Optimization

Reducing Memory Footprint:

```
# Disable structure bits for lower memory usage
```

```
foldseek search queryDB targetDB result tmp --sort-by-structure-bits 0
```

```
# Use sequence-only operations when coordinates not needed
```

```
foldseek search queryDB targetDB result tmp --alignment-type 0
```

Memory Usage Breakdown:	Component	Memory Usage	When to Optimize
----- -----	3Di sequences	~1 byte/residue	Always present
----- -----	Cα coordinates	~12 bytes/residue	TM-score calculations
----- -----	Secondary structure	~1 byte/residue	Structural alignment
----- -----	Index structures	~8 bytes/entry	Large databases
----- -----	Header data	Variable	Output formatting

10.3.2.2 Index Optimization Strategies

Index Subset Selection:

```
# For sequence-only searches (fastest)
```

```
foldseek createindex targetDB tmp --index-subset 1
```



```
# For structural searches (most comprehensive)
foldseek createindex targetDB tmp --index-subset 2
```

```
# For mixed workloads (balanced)
foldseek createindex targetDB tmp --index-subset 0
```

Index Memory Mapping:

```
// Load index only (no data) for prefiltering
IndexReader reader(dbPath, threads, IndexReader::INDEX_ONLY);

// Load data only (no index) for sequential access
DBReader<unsigned int> reader(dbPath, indexPath, threads, DBReader<unsigned
int>::USE_DATA);

// Load both for random access
DBReader<unsigned int> reader(dbPath, indexPath, threads,
                             DBReader<unsigned int>::USE_DATA |
                             DBReader<unsigned int>::USE_INDEX);
```

10.3.2.3 GPU Database Optimization

Memory Layout for GPU Processing:

```
# Create GPU-optimized database
foldseek makepaddeddb targetDB targetDB_gpu --padding-mode 1

# Use optimized GPU search
foldseek search queryDB targetDB_gpu result tmp --gpu 1 --prefilter-mode 1
```

GPU Memory Management: - **Streaming:** Automatic handling of databases larger than GPU memory - **Batching:** Processes queries in batches for optimal GPU utilization - **Memory Pools:** Reuses GPU memory across multiple operations

10.3.3 Database Quality Assessment

10.3.3.1 Structural Data Validation

Coordinate Quality Checks:

```
# Validate Cα coordinates in database
foldseek validateca targetDB validation_report

# Check secondary structure assignments
foldseek validatess targetDB ss_report
```

Database Integrity Verification:

```
# Verify database consistency
foldseek dbverify targetDB integrity_report

# Check index consistency
foldseek indexverify targetDB index_report
```

10.3.3.2 Performance Benchmarking

Database Access Benchmarking:

```
# Benchmark random access performance
foldseek benchmark targetDB benchmark_results --access-pattern random
```

```
# Benchmark sequential access performance
foldseek benchmark targetDB benchmark_results --access-pattern sequential
```

Benchmark Results Interpretation: - **Random Access:** Tests index efficiency and memory mapping - **Sequential Access:** Tests streaming performance and I/O efficiency - **Mixed Access:** Tests real-world usage patterns

10.3.4 Database Troubleshooting and Maintenance

10.3.4.1 Common Database Issues

Index Corruption:

```
# Rebuild corrupted index
foldseek createindex targetDB tmp --force-rebuild
```

```
# Verify index integrity
foldseek indexverify targetDB verification_report
```

Memory Mapping Issues:

```
# Check system memory mapping limits
ulimit -a | grep -E "(memlock|vmemory)"
```

```
# Increase limits if needed
ulimit -l unlimited # locked memory
ulimit -v unlimited # virtual memory
```

Database File Permissions:

```
# Fix file permissions
chmod 644 targetDB*
chmod 644 targetDB*.index
chmod 644 targetDB*.lookup
```

10.3.4.2 Database Repair and Recovery

Repair Corrupted Database:

```
# Create backup first
cp -r targetDB targetDB.backup

# Attempt repair
foldseek dbrepair targetDB repair_log.txt

# Verify repair
foldseek dbverify targetDB verification_report
```

Recover from Partial Database:

```
# Extract valid entries
foldseek extractvalid targetDB partial_db

# Rebuild from valid entries
foldseek createdb partial_db/ rebuilt_db
```

10.3.4.3 Performance Monitoring

Database Access Monitoring:

```
# Monitor database access patterns
foldseek dbmonitor targetDB monitor.log --duration 3600

# Analyze access patterns
foldseek dbanalyze targetDB analysis_report
```

Memory Usage Analysis:

```
# Monitor memory usage during operations
foldseek memprofile search queryDB targetDB result tmp profile.log

# Analyze memory bottlenecks
foldseek memanalyze profile.log analysis.txt
```

10.3.4.4 Database Migration and Conversion

Version Migration:

```
# Migrate from older Foldseek version
foldseek dbmigrate old_format_db new_format_db

# Verify migration
foldseek dbverify new_format_db migration_report
```

Format Conversion:

```
# Convert between different structural formats
foldseek convertformat input_db output_db --input-format cif --output-format pdb

# Batch conversion
find /path/to/structures -name "*.cif" | foldseek convertformat - output_db
```

10.3.4.5 Database Maintenance Scripts

Automated Maintenance:

```
#!/bin/bash
# Daily database maintenance script

DB_PATH="/path/to/databases"
LOG_FILE="/var/log/foldseek_maintenance.log"

# Verify database integrity
```

```

for db in "$DB_PATH"/*; do
    if [ -d "$db" ] && [ -f "$db/targetDB.dbtype" ]; then
        echo "$(date): Verifying $db" >> "$LOG_FILE"
        foldseek dbverify "$db" "$db.verify.log"
    fi
done

# Clean temporary files
find "$DB_PATH" -name "*.tmp" -mtime +7 -delete

# Rebuild indices if needed
for db in "$DB_PATH"/*; do
    if [ -d "$db" ] && [ -f "$db/targetDB.dbtype" ]; then
        echo "$(date): Checking index for $db" >> "$LOG_FILE"
        foldseek indexverify "$db" "$db.index.log"
    fi
done

```

Maintenance Scheduling:

```

# Add to crontab for automated maintenance
crontab -e

# Add this line for daily maintenance at 2 AM
0 2 * * * /path/to/maintenance_script.sh

```

10.3.4.6 Database Backup and Recovery

Backup Strategy:

```

# Full database backup
tar -czf foldseek_databases_$(date +%Y%m%d).tar.gz /path/to/databases/

# Incremental backup
rsync -av --delete /path/to/databases/ /path/to/backup/databases/

# Cloud backup
aws s3 sync /path/to/databases/ s3://my-bucket/foldseek-databases/

```

Recovery Procedures:

```

# Restore from backup
tar -xzf foldseek_databases_20231201.tar.gz -C /path/to/databases/

# Verify restoration
foldseek dbverify /path/to/databases/* verification_report.txt

# Rebuild indices after restoration
for db in /path/to/databases/*; do
    if [ -d "$db" ]; then
        foldseek createindex "$db" tmp --force-rebuild
    fi
done

```

```
fi
done
```

10.3.4.7 Database Security

Access Control:

```
# Set appropriate permissions
chmod 750 /path/to/databases/
chmod 640 /path/to/databases/*/*

# Restrict access to specific users
chown -R foldseek_user:foldseek_group /path/to/databases/

# Set up access logging
foldseek dbmonitor /path/to/databases/ access.log --log-access
```

Encryption (if needed):

```
# Encrypt sensitive databases
gpg -c /path/to/databases/sensitive_db

# Decrypt for use
gpg -d /path/to/databases/sensitive_db.gpg > /tmp/decrypted_db
foldseek createdb /tmp/decrypted_db/ working_db
```

This comprehensive database management section provides users with the tools and knowledge needed to maintain, troubleshoot, and optimize their Foldseek databases for production use.

10.4 Structural Alignment Details

10.4.1 3Di Substitution Matrix

The 3Di alphabet uses a specialized substitution matrix optimized for structural similarity:

	H	E	C
H	10	-5	-3
E	-5	10	-3
C	-3	-3	5

This matrix reflects the structural relationships: - **H-H**: Strong preference for helical matches - **E-E**: Strong preference for strand matches - **C-C**: Moderate preference for coil matches - **H-E**: Penalty for helix-strand mismatches - **H-C**, **E-C**: Moderate penalties for secondary structure mismatches

10.4.2 TM-score Calculation

TM-score is calculated as:

$$\text{TM-score} = \max(1/L_{\text{target}} * \sum [1/(1 + (d_i/d_0)^2)] \text{ over all alignments})$$

Where: - L_{target} : Length of target protein - d_i : Distance between i -th pair of residues after superposition - d_0 : Distance threshold (0.5 nm for $C\alpha$)

10.4.3 LDDT Score Implementation

LDDT (Local Distance Difference Test) measures local structural conservation:

$$\text{LDDT} = (1/4) * \sum [\delta(d_i < 0.5) + \delta(d_i < 1.0) + \delta(d_i < 2.0) + \delta(d_i < 4.0)]$$

Where $\delta(x)$ is 1 if x is true, 0 otherwise.

10.5 Performance Optimization

10.5.1 GPU Acceleration Details

10.5.1.1 CUDA Implementation

Enable GPU acceleration

```
foldseek search queryDB targetDB resultDB tmp --gpu 1 --prefilter-mode 1
```

Multi-GPU usage

```
CUDA_VISIBLE_DEVICES=0,1 foldseek search queryDB targetDB resultDB tmp --gpu 1
```

10.5.1.2 Memory Management

- **GPU Memory:** ~4GB for typical searches
- **Host Memory:** ~8GB for data structures
- **Streaming:** Automatic handling of databases larger than GPU memory

10.5.2 Parallel Processing

10.5.2.1 Multi-threading

Optimize thread usage

```
foldseek search queryDB targetDB resultDB tmp --threads 32
```

MPI for multi-node processing

```
RUNNER="mpirun -np 64" foldseek search queryDB targetDB resultDB tmp
```

10.5.2.2 Database Sharding

Split large searches across multiple processes

```
foldseek splitdb queryDB queryDB_split --split 100
```

Process shards in parallel

```
parallel -j 50 "foldseek search queryDB_split_{}_100 targetDB result_{}_tmp_{}" ::: {0..99}
```

10.6 Advanced Workflows

10.6.1 Iterative Structural Refinement

First iteration with high sensitivity

```
foldseek search queryDB targetDB iter1 tmp -s 9.5 --num-iterations 1
```

Extract hits and create profile

```
foldseek result2structprofile queryDB targetDB iter1 profileDB
```

```
# Second iteration with profiles
foldseek search profileDB targetDB iter2 tmp --profile-search
```

10.6.2 Structure-Based Function Annotation

```
# Search against functionally annotated structures
foldseek search unknown.pdb annotated_structures.db functional_hits tmp

# Transfer functional annotations
foldseek convertalis unknown.pdb annotated_structures.db functional_hits
functional_annotation.tsv
```

10.6.3 Quality Assessment Integration

```
# Assess structure quality using LDDT
foldseek easy-search query.pdb reference.db lddt_scores tmp --format-output
"query,target,lddt,lddtfull"

# Filter high-quality alignments
foldseek filterdb lddt_scores high_quality --filter-expression '$3 > 0.7'
```

This expert manual provides the foundation for advanced Foldseek usage, enabling researchers to build sophisticated structural bioinformatics pipelines and optimize performance for large-scale structural analyses.

10.7 Technical Comparison with MMseqs2

10.7.1 Database Architecture Differences

10.7.1.1 MMseqs2 Database Structure

MMseqs2 Database Files:

- └─ database (Data File): Raw sequence data
- └─ database.index: Sequence index (id, offset, length)
- └─ database_h: FASTA headers
- └─ database_h.index: Header index
- └─ database.lookup: ID to FASTA identifier mapping
- └─ database.dbtype: Database type specification

10.7.1.2 Foldseek Database Structure

Foldseek Database Files:

- └─ database (Data File): 3Di sequences + amino acid sequences
- └─ database.index: Structural index (id, offset, length)
- └─ database_h: Structure headers + metadata
- └─ database_h.index: Header index
- └─ database_ca: Cα coordinates for TM-score calculations
- └─ database_ss: Secondary structure assignments
- └─ database.lookup: ID to structure identifier mapping
- └─ database.dbtype: Database type specification

10.7.2 Algorithmic Differences

10.7.2.1 Prefiltering Stage

MMseqs2 Prefiltering: - Uses amino acid k-mers (typically k=6-14) - Employs reduced alphabet for sensitivity - Double consecutive k-mer matches on same diagonal - Ungapped alignment scoring with sequence substitution matrices

Foldseek Prefiltering: - Uses 3Di k-mers (H/E/C alphabet) with 20 internal centroids for classification - Specialized 3Di substitution matrix: $\begin{matrix} & H & E & C \\ H & 10 & -5 & -3 \\ E & -3 & 10 & -3 \\ C & -3 & -3 & 5 \end{matrix}$ - **Internal classification:** 20 centroids (states 0-19) for detailed structural discrimination - **Output mapping:** Internal states mapped to 3-letter alphabet via substitution matrix - Structural diagonal detection - Ungapped structural alignment with 3Di scoring

10.7.2.2 Alignment Stage

MMseqs2 Alignment: - SIMD-accelerated Smith-Waterman algorithm - Sequence identity calculation: $\text{identical_positions} / \text{alignment_length}$ - E-value computation using Karlin-Altschul statistics - Memory usage: ~8 bytes per residue for dynamic programming matrices

Foldseek Alignment: - 3Di+AA combined alignment algorithm - Multiple alignment modes: - Mode 0: 3Di Gotoh-Smith-Waterman (local) - Mode 1: TM-align (global structural) - Mode 2: 3Di+AA Gotoh-Smith-Waterman (local, default) - TM-score calculation: $\max(1/L_{\text{target}} * \sum [1/(1 + (d_i/d_0)^2)])$ - LDDT scoring: $(1/4) * \sum [\delta(d_i < 0.5) + \delta(d_i < 1.0) + \delta(d_i < 2.0) + \delta(d_i < 4.0)]$ - Memory usage: ~1 byte per residue for 3Di sequences

10.7.3 Performance Characteristics

10.7.3.1 Memory Efficiency

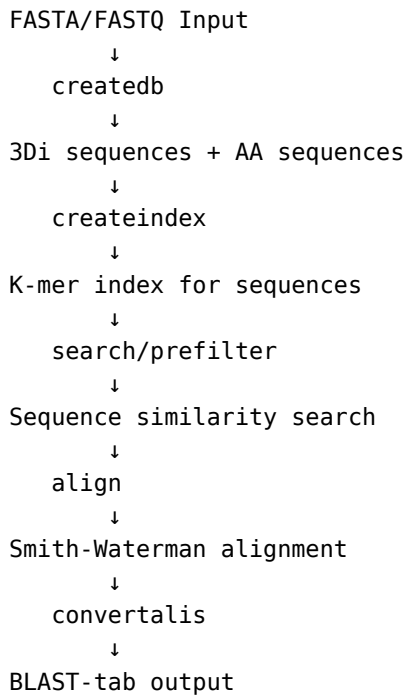
Database Size	MMseqs2 Memory	Foldseek Memory	Reduction Factor
UniProt (55M seq)	~71 GB	~35 GB	2.0x
AlphaFold (200M structures)	N/A	~200 GB	N/A
PDB (200K structures)	N/A	~1 GB	N/A

10.7.3.2 Speed Comparison

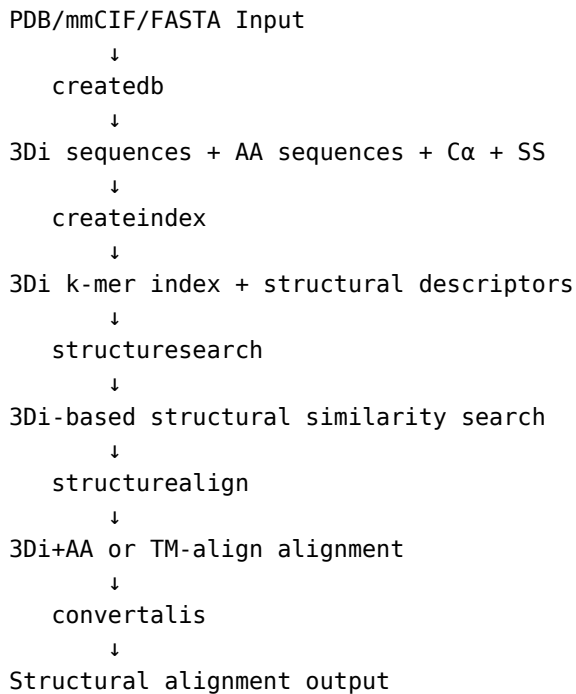
Operation	MMseqs2	Foldseek	Speedup
Prefiltering	10,000x vs BLAST	10,000x vs TM-align	Similar
Database Creation	Minutes	Minutes	Similar
Index Creation	Minutes	Minutes	Similar
GPU Acceleration	Yes	Yes	Similar

10.7.4 Data Processing Pipeline Differences

10.7.4.1 MMseqs2 Pipeline



10.7.4.2 Foldseek Pipeline



10.7.5 Advanced Features Comparison

10.7.5.1 MMseqs2 Advanced Features

- Iterative profile searches (PSI-BLAST style)
- Translated searches (BLASTX, TBLASTN, TBLASTX)
- Taxonomy assignment with LCA
- Profile-profile searches
- GPU acceleration for sequence search

10.7.5.2 Foldseek Advanced Features

- Direct structure-to-structure alignment
- TM-score based global alignment
- LDDT-based local structural conservation
- Multi-chain complex analysis
- Structure-based clustering
- GPU acceleration for structural search
- 3Di prediction from sequences (ProstT5)

10.7.6 Integration and Compatibility

10.7.6.1 Shared Infrastructure

- Both use memory-mapped I/O for efficient database access
- Compatible database format extensions
- Shared modular architecture
- Common indexing strategies
- Similar parameter interfaces

10.7.6.2 Unique Integration Points

- Foldseek extends MMseqs2's IndexReader for structural data
- Compatible with MMseqs2's taxonomy system
- Can use MMseqs2's clustering algorithms
- Shared result processing and output formatting

This technical comparison highlights how Foldseek builds upon MMseqs2's proven architecture while introducing structural biology-specific optimizations and algorithms.

11 Foldseek Developer Manual: Architecture and Module Classification

11.1 Overview

Foldseek is built on top of the MMseqs2 framework and extends it with structural biology capabilities. It is organized into hierarchical layers based on abstraction and functionality, similar to MMseqs2 but specialized for structural data processing.

11.2 Architecture Overview

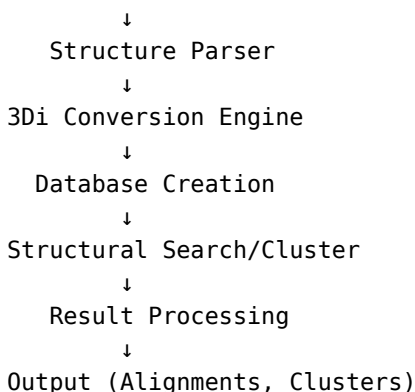
11.2.1 Core Components

Foldseek consists of several key architectural components:

1. **3Di Engine**: Converts protein structures to 3D-interaction sequences
2. **Structural Database Layer**: Manages structural data storage and retrieval
3. **Alignment Engine**: Performs structural alignments using multiple algorithms
4. **Search Engine**: Combines prefiltering and alignment for fast structural search
5. **Clustering Engine**: Groups structures by structural similarity

11.2.2 Data Flow Architecture

Input Structures (PDB/mmCIF)



11.3 Module Classification

11.3.1 Easy Workflows

High-level user-facing workflows that provide convenient interfaces for common structural analysis tasks.

- **easy-search**: Sensitive structural homology search workflow
- **easy-cluster**: Structure-based clustering workflow
- **easy-multimersearch**: Protein complex search workflow
- **easy-multimercluster**: Protein complex clustering workflow

11.3.2 Structure Search Modules

Core modules for structural similarity searching and alignment.

- **structuresearch**: Main structural search combining prefiltering and alignment
- **structurealign**: Direct structural alignment between structure sets
- **talign**: TM-align based global structural alignment
- **aln2tmscore**: Convert alignments to TM-scores and transformations
- **structureungappedalign**: Fast ungapped structural alignment

11.3.3 Structure Clustering Modules

Modules for clustering structures based on similarity.

- **structurecluster**: Main structure-based clustering workflow
- **cluster**: Core clustering algorithm for structural data
- **structureeasyrbh**: Reciprocal best hit for structural data

11.3.4 Multimer Analysis Modules

Specialized modules for protein complex analysis.

- **multimersearch**: Core multimer search functionality
- **multimercluster**: Core multimer clustering functionality
- **scoremultimer**: Score multimer alignments
- **filtermultimer**: Filter multimer results
- **createmultimerreport**: Generate complex alignment reports
- **expandmultimer**: Expand multimer search results

11.3.5 Structure Manipulation Modules

Modules for processing and converting structural data.

- **createdb**: Create structural databases from PDB/mmCIF files
- **structureto3didescrptor**: Convert structures to 3Di descriptors
- **convert2pdb**: Convert alignment results to superimposed PDB files
- **compressca**: Compress C α coordinate data
- **createstructsubdb**: Create structural sub-databases
- **lolalign**: Local structural alignment

11.3.6 Database Management Modules

Modules for managing structural databases.

- **databases**: Download pre-built structural databases
- **createindex**: Create precomputed indices for structural searches
- **makepaddeddb**: Create GPU-optimized padded databases
- **structureindex**: Structural indexing functionality

11.4 Hierarchical Dependencies

11.4.1 Easy Workflows → Core Modules

Easy workflows call core modules to perform their tasks:

- **easy-search** internally calls **structuresearch** workflow
- **easy-cluster** internally calls **structurecluster** workflow
- **easy-multimersearch** internally calls **multimersearch** workflow
- **easy-multimercluster** internally calls **multimercluster** workflow

11.4.2 Core Modules → Structural Engines

Core modules use specialized structural engines:

- **structuresearch** calls 3Di prefiltering followed by structural alignment
- **structurecluster** calls structural search followed by clustering algorithms

- multimersearch calls complex matching followed by interface analysis
- talign calls TM-score optimization algorithms

11.4.3 Structural Engines → Data Access Layer

Structural engines use the data access layer for efficient I/O:

- All modules use DBReader for memory-mapped structural database access
- Coordinate transformations use optimized linear algebra routines
- 3Di processing uses SIMD-accelerated pattern matching

11.5 3Di Engine Architecture: Two-Tier System

11.5.1 3Di Conversion Pipeline

```
// Core 3Di conversion process
StructureParser parser(inputFile);
SecondaryStructureAssigner ssAssigner;
CoordinateProcessor coordProcessor;
ThreeDiEncoder encoder;

// Process each structure
for each structure in input:
    ssAssigner.assignSecondaryStructure(structure);
    coordProcessor.extractCAlpha(structure);
    encoder.generate3DiSequence(structure);
```

11.5.2 Secondary Structure Assignment

The 3Di engine uses a multi-step process for secondary structure assignment:

1. **DSSP Algorithm:** Computes hydrogen bond patterns
2. **Geometric Analysis:** Analyzes C α backbone geometry
3. **Context Smoothing:** Reduces noise in assignments
4. **3Di Encoding:** Maps to H/E/C alphabet

11.5.3 Two-Tier 3Di System

11.5.3.1 Tier 1: Internal Classification (20 States)

- **Location:** foldseek/lib/3di/structureto3di.cpp, discretizeEmbeddings() function (lines 233-255)
- **Process:** Assigns each residue to closest of 20 centroids using Euclidean distance
- **Code:**

```
void StructureTo3Di::discretizeEmbeddings(std::vector<char> & states,
std::vector<Embedding> & embeddings,
std::vector<bool> & mask, const size_t
len){
    for (size_t i = 0; i < len; i++){
        if (mask[i]){
            // Find closest of 20 centroids
            for (size_t j = 0; j < Alphabet3Di::CENTROID_CNT; j++){ //
```

```

CENTROID_CNT = 20
    double sum = 0.0;
    for (size_t k = 0; k < Alphabet3Di::EMBEDDING_DIM; k++){
        sum += pow(embeddings[i].f[k] - Alphabet3Di::centroids[j][k],
2);
    }
    if (sum < minDistance){
        closestState = j; // States 0-19
        minDistance = sum;
    }
}
states[i] = closestState; // Internal states 0-19
}
}

```

11.5.3.2 Tier 2: External Mapping (3-Letter Alphabet)

- **Location:** foldseek/src/strucclustutils/structcreatedb.cpp (lines 518-530)
- **Process:** Maps internal states (0-19) to 3-letter alphabet (H/E/C)
- **Code:**

```

char * states = structureTo3Di.structure2states(&readStructure.ca[chainStart],
                                                &readStructure.n[chainStart],
                                                &readStructure.c[chainStart],
                                                &readStructure.cb[chainStart],
                                                chainLen);

// Map internal states (0-19) to 3-letter alphabet
alphabet3di.push_back(mat.num2aa[static_cast<int>(states[pos])]);

```

11.5.3.3 Internal vs. External Alphabet

The 3Di encoding process uses a two-stage classification system:

Internal Classification (20 states): - Uses **20 trained centroids** for detailed structural classification - Each centroid represents a cluster of similar structural features - Provides rich discrimination between different structural conformations - States are numbered 0-19 for internal processing

External Representation (3 letters): - Maps internal states to 3-letter alphabet (H/E/C) - Uses substitution matrix lookup: output = mat3Di.num2aa[internal_state] - Enables efficient k-mer indexing and alignment - Reduces computational complexity from 20×20 to 3×3 operations

This design achieves the optimal balance between: - **Classification accuracy** (20 internal states) - **Computational efficiency** (3-letter output) - **Memory usage** (~1 byte per residue vs ~8 bytes for coordinates)

11.5.4 AA and 3Di Integration: Combined Storage and Usage

11.5.4.1 Storage Architecture

From `foldseek/src/strucclustutils/structcreatedb.cpp` (lines 518-570), AA and 3Di sequences are stored together but in separate coordinated database files:

```
// Generate both sequences for each structure
char * states = structureTo3Di.structure2states(&readStructure.ca[chainStart],
                                                &readStructure.n[chainStart],
                                                &readStructure.c[chainStart],
                                                &readStructure.cb[chainStart],
                                                chainLen);

// Store 3Di sequence (H/E/C alphabet)
for (size_t pos = 0; pos < chainLen; pos++) {
    alphabet3di.push_back(mat.num2aa[static_cast<int>(states[pos])]);
}

// Store AA sequence (20-letter alphabet)
for (size_t pos = 0; pos < chainLen; pos++) {
    alphabetAA.push_back(readStructure.ami[chainStart+pos]);
}

// Write to separate but coordinated database files
torsiondbw.writeData(alphabet3di.data(), alphabet3di.size(), dbKey,
thread_idx); // 3Di sequences
aadbw.writeData(alphabetAA.data(), alphabetAA.size(), dbKey,
thread_idx); // AA sequences
```

11.5.4.2 Database File Structure

File	Content	Size Impact	Purpose
database	Amino acid sequences (20-letter alphabet)	~1 byte/residue	Sequence-based operations
database_ss	3Di sequences (H/E/C alphabet)	~1 byte/residue	Structural operations
database_ca	C α coordinates	~12 bytes/residue	TM-score calculations
database_h	Headers/metadata	Variable	Structure identification

11.5.4.3 Combined Alignment Process

From `foldseek/src/strucclustutils/structurealign.cpp`, AA and 3Di are combined during alignment:

```

// Initialize alignment with both sequence types
StructureSmithWaterman structureSmithWaterman(par.maxSeqLen,
subMat3Di.alphabetSize,
par.compBiasCorrection,
par.compBiasCorrectionScale,
&subMatAA, &subMat3Di);

// Use both sequences in alignment scoring
StructureSmithWaterman::s_align align =
structureSmithWaterman.alignScoreEndPos<StructureSmithWaterman::PROFILE>(
tSeqAA.numSequence, tSeq3Di.numSequence, targetSeqLen,
par.gapOpen.values.aminoacid(), par.gapExtend.values.aminoacid(),
querySeqLen / 2
);

// Weighting: 3Di+AA alignment uses both scoring matrices
float aaFactor = (par.alignmentType == LocalParameters::ALIGNMENT_TYPE_3DI_AA) ?
1.4 : 0.0;
SubstitutionMatrix subMatAA(blosum.c_str(), aaFactor, par.scoreBias);

```

11.5.4.4 Key Implementation Details

1. **Synchronized Storage:** Both sequences have identical:
 - Length (same number of residues)
 - Database keys (same identifiers)
 - Indexing (same positions in database files)
2. **Combined Scoring:** The 3Di+AA alignment uses:
 - **3Di substitution matrix** for structural similarity
 - **AA substitution matrix** (BLOSUM62) for sequence similarity
 - **Weighted combination** (AA factor = 1.4 for 3Di+AA mode)
3. **Module Usage Patterns:**
 - **Search operations:** Load both AA and 3Di for combined scoring
 - **Structure-only operations:** Load only 3Di sequences
 - **Sequence-only operations:** Load only AA sequences
 - **TM-score calculations:** Load C α coordinates + sequences

11.5.5 SIMD Optimization

Foldseek uses SIMD instructions for performance-critical operations:

```

// SIMD-accelerated 3Di alignment
__m256i query_3di = _mm256_loadu_si256((__m256i*)query_ptr);
__m256i target_3di = _mm256_loadu_si256((__m256i*)target_ptr);
__m256i score = _mm256_subs_epi8(query_3di, target_3di);

```

11.6 Structural Database Layer

11.6.1 Database File Architecture

Foldseek extends the MMseqs2 database format with structural data components:

11.6.1.1 Database File Components

File	Content	Description/Size/Access
database	Concatenated 3Di sequences, amino acid sequences, and structural descriptors	Main data file ~2 bytes/residue Sequential/Indexed
database.index	Tab-separated: id\toffset\tlength for random access	Index file Minimal Memory-mapped
database_ca	Binary C α coordinates for TM-score calculations	C α coordinates ~12 bytes/residue Random access
database_ca.index	Index for coordinate data access	C α index Minimal Memory-mapped
database_ss	3Di sequences (H/E/C) and structural assignments	Secondary structure ~1 byte/residue Sequential/Indexed
database_ss.index	Index for secondary structure data	SS index Minimal Memory-mapped
database_h	FASTA headers and structural metadata	Header database Variable Random access
database_h.index	Index for header data access	Header index Minimal Memory-mapped
database.lookup	Maps internal numeric IDs to original structure identifiers	Lookup table Minimal Hash lookup
database.dbtype	Binary file specifying data types (type 10 for structural)	Database type 4 bytes Read-once

11.6.1.2 Index Structure and Memory Mapping

Foldseek uses memory-mapped I/O for efficient database access:

```
// Memory-mapped database access pattern
DBReader<unsigned int> sequenceReader(dbPath, indexPath, threads,
                                     DBReader<unsigned int>::USE_DATA |
                                     DBReader<unsigned int>::USE_INDEX |
```

```

                                DBReader<unsigned int>::USE_HEADER);
DBReader<unsigned int> caReader(caPath, caIndexPath, threads,
                                DBReader<unsigned int>::USE_DATA |
                                DBReader<unsigned int>::USE_INDEX);
DBReader<unsigned int> ssReader(ssPath, ssIndexPath, threads,
                                DBReader<unsigned int>::USE_DATA |
                                DBReader<unsigned int>::USE_INDEX);

```

11.6.1.3 Index File Format

The index files follow the MMseqs2 format optimized for structural data:

database.index format:

id	offset	length	
0	0	1247	# First structure: 1247 bytes
1	1247	892	# Second structure: 892 bytes
2	2139	1456	# Third structure: 1456 bytes
...			

Where: - id: Internal numeric identifier (0-based, sequential) - offset: Byte offset in the data file

- length: Length of the entry in bytes

11.6.1.4 Structural Index Architecture

Foldseek creates specialized indices for different types of structural queries:

```

// 3Di k-mer index for structural search
struct StructuralIndex {
    std::vector<uint64_t> kmerTable;           // 3Di k-mer hash table
    std::vector<uint32_t> positionTable;       // Position lookup table
    std::vector<float> scoreMatrix;           // 3Di substitution scores
    std::vector<char> sequenceData;           // 3Di sequences for fast access
};

// Index creation for different subsets
enum IndexSubset {
    INDEX_SUBSET_ALL = 0,                    // All data types
    INDEX_SUBSET_SEQUENCE = 1,               // Sequence data only
    INDEX_SUBSET_STRUCTURE = 2               // Structural data only
};

```

11.6.1.5 Database Type Specifications

The database.dbtype file contains binary type information:

```

enum DatabaseType {
    DBTYPE_GENERIC_DB = 0,                  // Basic sequence data
    DBTYPE_AMINO_ACID = 1,                  // Protein sequences
    DBTYPE_NUCLEOTIDE = 2,                  // DNA/RNA sequences
    DBTYPE_STRUCTURAL_DB = 10,              // 3Di + coordinates + secondary structure
    DBTYPE_PROFILE_DB = 11,                 // PSSM data
    DBTYPE_ALIGNMENT_DB = 12                // Alignment results
};

```

11.6.1.6 Memory Mapping Strategy

Foldseek uses memory-mapped files for instant access to any structure:

```
// Example: Accessing structural data by ID
size_t entryId = 42;
const char* sequenceData = sequenceReader.getData(entryId, thread_idx);
const char* caData = caReader.getData(entryId, thread_idx);
const char* ssData = ssReader.getData(entryId, thread_idx);

// Memory-efficient access patterns
enum AccessMode {
    USE_DATA = 1,          // Load data file
    USE_INDEX = 2,         // Load index file
    USE_HEADER = 4,        // Load header data
    USE_LOOKUP = 8         // Load lookup table
};
```

This architecture enables: - **Random access** to millions of structures without loading entire database - **Parallel processing** across multiple cores - **Efficient caching** by the operating system - **Memory efficiency** - only active data is resident in RAM

11.7 Alignment Engine Architecture

11.7.1 Multiple Alignment Algorithms

Foldseek supports multiple alignment algorithms:

1. **3Di+AA Alignment**: Combines structural and sequence information
2. **TM-align**: Global structural alignment
3. **Local Structural Alignment**: Smith-Waterman on 3Di sequences

11.7.2 TM-score Implementation

The TM-score engine implements the TM-align algorithm:

```
// Core TM-score calculation
float calculateTMScore(const std::vector<CAlpha>& query,
                      const std::vector<CAlpha>& target,
                      const Matrix3x3& rotation,
                      const Vector3& translation) {
    float score = 0.0f;
    float d0 = 1.24f * pow(query.size() - 15.0f, 1.0f/3.0f) - 1.8f;
    for (size_t i = 0; i < query.size(); ++i) {
        float distance = calculateDistance(query[i], target[i], rotation,
translation);
        score += 1.0f / (1.0f + pow(distance / d0, 2.0f));
    }
    return score / query.size();
}
```

11.7.3 SIMD Alignment Acceleration

Alignment operations use SIMD instructions for performance:

```

// SIMD-accelerated dynamic programming
void alignSIMD(const char* query, const char* target,
               __m256i* scoreMatrix, __m256i* tracebackMatrix) {
    // Vectorized alignment computation
    for (size_t i = 1; i < queryLen; ++i) {
        for (size_t j = 1; j < targetLen; ++j) {
            // SIMD operations for score calculation
        }
    }
}

```

11.8 Multimer Engine Architecture

11.8.1 Complex Representation

Multimer structures are represented as collections of chains:

```

struct Multimer {
    std::vector<Chain> chains;
    std::vector<Interface> interfaces;
    Matrix4x4 transformation;
    float complexScore;
};

```

11.8.2 Chain Matching Algorithm

The multimer engine uses optimal assignment algorithms:

```

// Hungarian algorithm for chain matching
std::vector<int> matchChains(const std::vector<Chain>& queryChains,
                             const std::vector<Chain>& targetChains) {
    // Compute similarity matrix
    Matrix similarityMatrix = computeChainSimilarities(queryChains,
                                                         targetChains);
    // Apply Hungarian algorithm
    return hungarianAlgorithm(similarityMatrix);
}

```

11.8.3 Interface Analysis

Interface analysis computes interaction scores:

```

// Interface LDDT calculation
float calculateInterfaceLDDT(const Interface& interface,
                             const Matrix4x4& transformation) {
    float totalScore = 0.0f;
    for (const auto& contact : interface.contacts) {
        float distance = calculateContactDistance(contact, transformation);
        totalScore += lddtScore(distance);
    }
    return totalScore / interface.contacts.size();
}

```

11.9 GPU Acceleration Architecture

11.9.1 CUDA Integration

Foldseek integrates CUDA for GPU acceleration:

```
// CUDA kernel for structural alignment
__global__ void structuralAlignmentKernel(const float* queryCoords,
                                         const float* targetCoords,
                                         float* scoreMatrix,
                                         int querySize, int targetSize) {
    // GPU-accelerated alignment computation
}
```

11.9.2 Memory Management

GPU memory is managed efficiently:

```
// GPU memory allocation and transfer
void allocateGPUResources(const StructuralDatabase& db) {
    // Allocate GPU memory for coordinates
    cudaMalloc(&gpuQueryCoords, querySize * sizeof(float3));
    cudaMalloc(&gpuTargetCoords, targetSize * sizeof(float3));
    // Transfer data to GPU
    cudaMemcpy(gpuQueryCoords, hostQueryCoords, querySize * sizeof(float3),
               cudaMemcpyHostToDevice);
}
```

11.10 Performance Optimization

11.10.1 Parallel Processing

Foldseek uses multiple levels of parallelism:

1. **Thread-level:** OpenMP for multi-core processing
2. **SIMD-level:** Vector instructions for data-parallel operations
3. **GPU-level:** CUDA for massively parallel computations
4. **MPI-level:** Multi-node processing for large-scale analyses

11.10.2 Memory Optimization

Memory usage is optimized through:

1. **Memory-mapped I/O:** Avoids loading entire databases into RAM
2. **Streaming processing:** Processes data in chunks
3. **Compression:** Compresses structural data when possible
4. **Caching:** Intelligent caching of frequently accessed data

11.10.3 Algorithmic Optimization

Key optimizations include:

1. **Early termination:** Stops computation when results are unlikely to improve
2. **Pruning:** Eliminates unpromising candidates early

3. **Approximation:** Uses fast approximations for initial scoring
4. **Indexing:** Precomputed indices for fast lookup

11.11 Development Guidelines

11.11.1 Code Organization

Foldseek follows a modular architecture:

```
src/
├── commons/           # Shared utilities and data structures
├── alignment/        # Alignment algorithms
├── clustering/        # Clustering algorithms
├── prefiltering/      # Prefiltering algorithms
├── strucclustutils/   # Structural clustering utilities
├── workflow/         # High-level workflow implementations
└── version/          # Version information
```

11.11.2 Testing Strategy

Comprehensive testing includes:

1. **Unit tests:** Individual function testing
2. **Integration tests:** Module interaction testing
3. **Performance tests:** Benchmarking against known datasets
4. **Regression tests:** Ensuring compatibility with previous versions

11.11.3 Performance Profiling

Profiling tools are used to identify bottlenecks:

1. **gprof:** CPU profiling
2. **nvprof:** GPU profiling
3. **perf:** System-level profiling
4. **Custom timers:** Module-specific timing

This hierarchical architecture allows for modular development, easy extension of Foldseek's capabilities, and efficient processing of large-scale structural data.